StreamTest AI

AI-POWERED TESTING & QUALITY ASSURANCE FOR LIVE STREAMING SYSTEMS

Production configurations, Relational database schemas,
Next.js 15 pages, and Interactive Sandbox APIs

Compiled by Antigravity AI Assistant

Formatted for GitHub Documentation

File: db/schema.sql

```
-- PostgreSQL Database Schema for StreamTest AI

-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Role Types
CREATE TYPE user_role AS ENUM ('admin', 'qa_engineer', 'developer', 'viewer');

-- Platform Types
CREATE TYPE channel_platform AS ENUM ('youtube', 'facebook', 'instagram', 'twitch', 'tiktok', 'linkedin');

-- Stream Status
CREATE TYPE stream_status AS ENUM ('scheduled', 'streaming', 'completed', 'failed', 'retrying');

-- Test Status
CREATE TYPE test_status AS ENUM ('passed', 'failed', 'running', 'queued');

-- Subscription Plan Types
CREATE TYPE plan_type AS ENUM ('free_trial', 'standard', 'premium', 'pro');

-- Subscription Status
CREATE TYPE subscription_status AS ENUM ('active', 'past_due', 'canceled', 'unpaid');

-------------------------------------------------------
-- 1. Users Table
-------------------------------------------------------
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255), -- Nullable to allow OAuth-only logins
    phone_number VARCHAR(50) UNIQUE, -- For Phone number OTP login
    google_id VARCHAR(100) UNIQUE, -- For Google OAuth integrations
    github_id VARCHAR(100) UNIQUE, -- For GitHub OAuth integrations
    otp_code VARCHAR(10), -- Temporary SMS verification OTP code
    otp_expires_at TIMESTAMP WITH TIME ZONE,
    role user_role DEFAULT 'qa_engineer',
    is_verified BOOLEAN DEFAULT FALSE,
    is_banned BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_phone ON users(phone_number);
CREATE INDEX idx_users_oauth ON users(google_id, github_id);

-------------------------------------------------------
-- 2. Subscriptions Table
-------------------------------------------------------
CREATE TABLE subscriptions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    plan plan_type NOT NULL DEFAULT 'free_trial',
    status subscription_status NOT NULL DEFAULT 'active',
    channel_limit INTEGER NOT NULL DEFAULT 1,
    storage_limit_gb DECIMAL(10,2) NOT NULL DEFAULT 5.00,
    stream_limit_hours INTEGER NOT NULL DEFAULT 10,
    current_period_start TIMESTAMP WITH TIME ZONE NOT NULL,
    current_period_end TIMESTAMP WITH TIME ZONE NOT NULL,
    cancel_at_period_end BOOLEAN DEFAULT FALSE,
```

```
    stripe_subscription_id VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_subscriptions_user_id ON subscriptions(user_id);

-- ------------------------------------------------------
-- 3. Billing History Table
-- ------------------------------------------------------
CREATE TABLE billing_history (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    subscription_id UUID REFERENCES subscriptions(id) ON DELETE SET NULL,
    amount DECIMAL(10,2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'USD',
    status VARCHAR(50) NOT NULL, -- 'paid', 'failed', 'refunded'
    invoice_pdf_url VARCHAR(512),
    tax_amount DECIMAL(10,2) DEFAULT 0.00,
    payment_gateway VARCHAR(50) DEFAULT 'stripe', -- 'stripe', 'paypal', 'razorpay'
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- ------------------------------------------------------
-- 4. Channel Integrations Table
-- ------------------------------------------------------
CREATE TABLE connected_channels (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    platform channel_platform NOT NULL,
    channel_name VARCHAR(255) NOT NULL,
    external_channel_id VARCHAR(255) NOT NULL,
    oauth_access_token TEXT NOT NULL,
    oauth_refresh_token TEXT,
    token_expires_at TIMESTAMP WITH TIME ZONE,
    permission_scopes TEXT[],
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_user_platform_channel UNIQUE (user_id, platform, external_channel_id)
);

CREATE INDEX idx_connected_channels_user ON connected_channels(user_id);

-- ------------------------------------------------------
-- 5. Videos Table (Library Management)
-- ------------------------------------------------------
CREATE TABLE videos (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    file_name VARCHAR(255) NOT NULL,
    file_size_bytes BIGINT NOT NULL,
    duration_seconds INTEGER NOT NULL,
    video_codec VARCHAR(50),
    audio_codec VARCHAR(50),
    resolution VARCHAR(20),
    storage_url VARCHAR(512) NOT NULL,
    status VARCHAR(50) DEFAULT 'ready', -- 'uploading', 'processing', 'ready', 'failed', 'corrupt'
    folder_path VARCHAR(255) DEFAULT '/', -- Supporting folder structure
    is_deleted BOOLEAN DEFAULT FALSE,
    deleted_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE INDEX idx_videos_user_folder ON videos(user_id, folder_path);

-- ------------------------------------------------------
-- 6. Streams Table (Live Stream Engine Scheduling)
-- ------------------------------------------------------
CREATE TABLE streams (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    video_id UUID REFERENCES videos(id) ON DELETE RESTRICT,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    scheduled_start TIMESTAMP WITH TIME ZONE NOT NULL,
    scheduled_end TIMESTAMP WITH TIME ZONE NOT NULL,
    actual_start TIMESTAMP WITH TIME ZONE,
    actual_end TIMESTAMP WITH TIME ZONE,
    status stream_status DEFAULT 'scheduled',
    retry_count INTEGER DEFAULT 0,
    max_retries INTEGER DEFAULT 3,
    stream_bitrate_kbps INTEGER DEFAULT 4500,
    fps INTEGER DEFAULT 30,
    resolution VARCHAR(20) DEFAULT '1080p',
    ffmpeg_pid INTEGER,
    worker_node_id VARCHAR(100),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_streams_schedule ON streams(scheduled_start, status);

-- ------------------------------------------------------
-- 7. Stream Destinations Table (Multicasting mappings)
-- ------------------------------------------------------
CREATE TABLE stream_destinations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    stream_id UUID REFERENCES streams(id) ON DELETE CASCADE,
    channel_id UUID REFERENCES connected_channels(id) ON DELETE CASCADE,
    rtmp_url VARCHAR(512) NOT NULL,
    stream_key VARCHAR(255) NOT NULL,
    status VARCHAR(50) DEFAULT 'pending', -- 'pending', 'streaming', 'completed', 'failed'
    error_message TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- ------------------------------------------------------
-- 8. Test Cases & Suites Table
-- ------------------------------------------------------
CREATE TABLE test_suites (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    description TEXT,
    test_type VARCHAR(100) NOT NULL, -- 'auth', 'billing', 'streaming', 'security', 'performance'
    target_module VARCHAR(100) NOT NULL,
    spec_json JSONB NOT NULL, -- Holds generated test case steps
    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- ------------------------------------------------------
-- 9. Test Results Table
-- ------------------------------------------------------
CREATE TABLE test_results (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    suite_id UUID REFERENCES test_suites(id) ON DELETE CASCADE,
    triggered_by UUID REFERENCES users(id) ON DELETE SET NULL,
    status test_status NOT NULL,
    duration_ms INTEGER NOT NULL,
    success_rate DECIMAL(5,2),
    logs TEXT,
    error_summary TEXT,
    report_url VARCHAR(512),
    screenshot_urls TEXT[],
    metrics_json JSONB, -- Storing specific metrics like response times or CPU
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_test_results_suite ON test_results(suite_id);

-------------------------------------------------------
-- 10. Security Scans Table
-------------------------------------------------------
CREATE TABLE security_scans (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    triggered_by UUID REFERENCES users(id) ON DELETE SET NULL,
    status test_status NOT NULL,
    vulnerabilities_found JSONB DEFAULT '[]', -- List of finding items
    risk_score DECIMAL(4,2) DEFAULT 0.00,
    owasp_categories_scanned VARCHAR(50)[],
    target_endpoint VARCHAR(512) NOT NULL,
    logs TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-------------------------------------------------------
-- 11. Infrastructure Metrics Table
-------------------------------------------------------
CREATE TABLE infrastructure_metrics (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    node_id VARCHAR(100) NOT NULL,
    cpu_usage_percent DECIMAL(5,2) NOT NULL,
    ram_usage_percent DECIMAL(5,2) NOT NULL,
    storage_usage_percent DECIMAL(5,2) NOT NULL,
    redis_connected_clients INTEGER DEFAULT 0,
    redis_queue_length INTEGER DEFAULT 0,
    active_ffmpeg_workers INTEGER DEFAULT 0,
    network_in_mbps DECIMAL(10,2) DEFAULT 0.00,
    network_out_mbps DECIMAL(10,2) DEFAULT 0.00,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_infra_metrics_time ON infrastructure_metrics(created_at DESC);

-------------------------------------------------------
-- 12. Automated Coupons & Campaigns (Admin Workflows)
-------------------------------------------------------
CREATE TABLE coupons (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    code VARCHAR(50) UNIQUE NOT NULL,
    discount_type VARCHAR(20) NOT NULL, -- 'percentage', 'fixed'
    discount_value DECIMAL(10,2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'USD',
    max_redemptions INTEGER DEFAULT 100,
    times_redeemed INTEGER DEFAULT 0,
    expires_at TIMESTAMP WITH TIME ZONE,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);
```

File: Dockerfile

```
# Dockerfile for Next.js 15 Frontend and Mock Engine

# Stage 1: Install dependencies
FROM node:20-alpine AS deps
RUN apk add --no-cache libc6-compat
WORKDIR /app

# Copy package files
COPY package.json package-lock.json* ./
RUN npm ci

# Stage 2: Rebuild the source code
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .

# Environment variables for build
ENV NEXT_TELEMETRY_DISABLED=1
ENV NODE_ENV=production

RUN npm run build

# Stage 3: Production runner
FROM node:20-alpine AS runner
WORKDIR /app

ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

# Copy static assets and standalone build
COPY --from=builder /app/public ./public

# Set correct permission for prerender cache
RUN mkdir .next
RUN chown nextjs:nodejs .next

# Automatically leverage output traces to reduce image size
# https://nextjs.org/docs/advanced-features/output-file-tracing
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

USER nextjs

EXPOSE 3000

ENV PORT=3000
ENV HOSTNAME="0.0.0.0"

# server.js is created by next build when output: 'standalone' is configured
CMD ["node", "server.js"]
```

File: docker-compose.yml

```
version: '3.8'

services:
  # Next.js Web Dashboard
  web:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://streamtest:streamtest_secure_pass@db:5432/streamtest_db?sslmode=disable
      - REDIS_URL=redis://redis:6379/0
      - NODE_ENV=production
    depends_on:
      - db
      - redis
    networks:
      - streamtest-network
    restart: always

  # PostgreSQL Database
  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_USER=streamtest
      - POSTGRES_PASSWORD=streamtest_secure_pass
      - POSTGRES_DB=streamtest_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      # Mount the schema directory to automatically initialize the DB
      - ./db/schema.sql:/docker-entrypoint-initdb.d/schema.sql
    networks:
      - streamtest-network
    restart: always
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U streamtest -d streamtest_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Redis for Queues (BullMQ) & Caching
  redis:
    image: redis:7-alpine
    command: redis-server --appendonly yes --requirepass redis_secure_pass
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    networks:
      - streamtest-network
    restart: always
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "redis_secure_pass", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  # NestJS API Backend (Production Microservice Target)
```

```
  backend-api:
    image: node:20-alpine
    working_dir: /app
    command: sh -c "echo 'NestJS service starting...' && tail -f /dev/null"
    environment:
      - DATABASE_URL=postgresql://streamtest:streamtest_secure_pass@db:5432/streamtest_db?sslmode=disable
      - REDIS_URL=redis://redis_secure_pass@redis:6379/0
      - JWT_SECRET=super_secret_jwt_signkey_102938
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - streamtest-network
    restart: always

  # FFmpeg Stream Worker (Queue Listener)
  stream-worker:
    image: jrottenberg/ffmpeg:5.1-alpine
    entrypoint: ["sh", "-c", "echo 'Streaming queue listener listening to BullMQ on Redis...' && tail -f /dev/null"]
    depends_on:
      - redis
    networks:
      - streamtest-network
    restart: always

networks:
  streamtest-network:
    driver: bridge

volumes:
  postgres_data:
    driver: local
  redis_data:
    driver: local
```

File: k8s/deployment.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: streamtest-ai
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: streamtest-config
  namespace: streamtest-ai
data:
  NODE_ENV: "production"
  DATABASE_URL: "postgresql://streamtest:streamtest_secure_pass@postgres-service:5432/streamtest_db"
  REDIS_URL: "redis://:redis_secure_pass@redis-service:6379/0"
---
apiVersion: v1
kind: Secret
metadata:
  name: streamtest-secrets
  namespace: streamtest-ai
type: Opaque
data:
  # Base64 encoded secrets
  JWT_SECRET: c3VwZXJfc2VjcmV0X2p3dF9zaWdua2V5XzEwMjkzOA== # super_secret_jwt_signkey_102938
  STRIPE_SECRET_KEY: c2tfX21vY2tfZGFzaGJvYXJkX3NlY3JldF9mb3JfdGVzdGluZ19vbmx5 #
sk__mock_dashboard_secret_for_testing_only
---
# Redis Service & Deployment
apiVersion: v1
kind: Service
metadata:
  name: redis-service
  namespace: streamtest-ai
spec:
  ports:
    - port: 6379
  selector:
    app: redis
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-deployment
  namespace: streamtest-ai
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          command: ["redis-server", "--requirepass", "redis_secure_pass"]
          ports:
            - containerPort: 6379
          resources:
```

```
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
---
# PostgreSQL Service & StatefulSet
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
  namespace: streamtest-ai
spec:
  ports:
    - port: 5432
  selector:
    app: postgres
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-statefulset
  namespace: streamtest-ai
spec:
  serviceName: "postgres-service"
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15-alpine
          env:
            - name: POSTGRES_USER
              value: "streamtest"
            - name: POSTGRES_PASSWORD
              value: "streamtest_secure_pass"
            - name: POSTGRES_DB
              value: "streamtest_db"
          ports:
            - containerPort: 5432
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "1"
              memory: "1Gi"
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: postgres-storage
      spec:
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
```

```
            storage: 10Gi
---
# Next.js Frontend Web Service & Deployment
apiVersion: v1
kind: Service
metadata:
  name: web-service
  namespace: streamtest-ai
spec:
  ports:
    - port: 80
      targetPort: 3000
  selector:
    app: web
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: streamtest-ai
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: gcr.io/streamtest-ai/frontend:latest
          ports:
            - containerPort: 3000
          envFrom:
            - configMapRef:
                name: streamtest-config
          env:
            - name: JWT_SECRET
              valueFrom:
                secretKeyRef:
                  name: streamtest-secrets
                  key: JWT_SECRET
            - name: STRIPE_SECRET_KEY
              valueFrom:
                secretKeyRef:
                  name: streamtest-secrets
                  key: STRIPE_SECRET_KEY
          readinessProbe:
            httpGet:
              path: /api/health
              port: 3000
            initialDelaySeconds: 15
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /api/health
```

```
              port: 3000
            initialDelaySeconds: 30
            periodSeconds: 20
          resources:
            requests:
              cpu: "200m"
              memory: "256Mi"
            limits:
              cpu: "1"
              memory: "512Mi"
---
# Horizontal Pod Autoscaler (HPA) for Web Frontend
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
  namespace: streamtest-ai
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 75
```

File: docs/ci-cd-pipelines.md

````
# CI/CD Pipeline Configurations

StreamTest AI integrates seamlessly with major CI/CD providers. Below are the complete configurations for GitHub Actions, GitLab CI/CD, and Jenkins.

---

## 1. GitHub Actions Workflow

Save this file as `.github/workflows/test.yml` in your repository.

```yaml
name: StreamTest AI CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

jobs:
  test:
    name: Build & Test
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_USER: streamtest
          POSTGRES_PASSWORD: streamtest_secure_pass
          POSTGRES_DB: streamtest_db
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      redis:
        image: redis:7-alpine
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install Dependencies
````
````

```
      run: npm ci

    - name: Lint Code
      run: npm run lint

    - name: Run Unit & Integration Tests (Jest)
      run: npm run test:unit
      env:
        DATABASE_URL: postgresql://streamtest:streamtest_secure_pass@localhost:5432/streamtest_db
        REDIS_URL: redis://localhost:6379/0

    - name: Install Playwright Browsers
      run: npx playwright install --with-deps

    - name: Run E2E & Visual Regression Tests (Playwright)
      run: npm run test:e2e
      env:
        DATABASE_URL: postgresql://streamtest:streamtest_secure_pass@localhost:5432/streamtest_db
        REDIS_URL: redis://localhost:6379/0

    - name: Execute Security Vulnerability Audit
      run: npm audit --audit-level=high

    - name: Build Next.js Application
      run: npm run build
      env:
        NEXT_TELEMETRY_DISABLED: 1

build-and-push:
  name: Build Container & Deploy
  needs: test
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest

  steps:
    - name: Checkout Repository
      uses: actions/checkout@v4

    - name: Set up QEMU
      uses: docker/setup-qemu-action@v3

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login to DockerHub (or GCR/AWS ECR)
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}

    - name: Build and Push Docker Image
      uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: |
          streamtest/frontend:latest
          streamtest/frontend:${{ github.sha }}

    - name: Deploy to Kubernetes Cluster
      uses: azure/k8s-deploy@v5
      with:
        manifests: |
          k8s/deployment.yaml
```

```
          images: |
            streamtest/frontend:${{ github.sha }}
          namespace: streamtest-ai
```

---

2. GitLab CI Configuration

Save this file as `.gitlab-ci.yml` in your repository root.

```yaml
stages:
  - lint
  - test
  - build
  - deploy

variables:
  POSTGRES_USER: streamtest
  POSTGRES_PASSWORD: streamtest_secure_pass
  POSTGRES_DB: streamtest_db
  DATABASE_URL: "postgresql://streamtest:streamtest_secure_pass@postgres:5432/streamtest_db"
  REDIS_URL: "redis://redis:6379/0"

cache:
  paths:
    - node_modules/

default:
  image: node:20-alpine

lint_code:
  stage: lint
  script:
    - npm ci
    - npm run lint

unit_tests:
  stage: test
  services:
    - name: postgres:15-alpine
      alias: postgres
    - name: redis:7-alpine
      alias: redis
  script:
    - npm ci
    - npm run test:unit

e2e_tests:
  stage: test
  image: mcr.microsoft.com/playwright:v1.40.0-jammy
  services:
    - name: postgres:15-alpine
      alias: postgres
    - name: redis:7-alpine
      alias: redis
  script:
    - npm ci
    - npx playwright install --with-deps
    - npm run test:e2e

docker_build:
  stage: build
```

```
  image: docker:24.0.5
  services:
    - docker:24.0.5-dind
  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""
  before_script:
    - docker login -u "$CI_REGISTRY_USER" -p "$CI_REGISTRY_PASSWORD" $CI_REGISTRY
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker tag $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA $CI_REGISTRY_IMAGE:latest
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
    - docker push $CI_REGISTRY_IMAGE:latest
  only:
    - main

k8s_deploy:
  stage: deploy
  image: dtzar/helm-kubectl:latest
  script:
    - kubectl config set-cluster k8s --server="$K8S_SERVER" --insecure-skip-tls-verify=true
    - kubectl config set-credentials admin --token="$K8S_TOKEN"
    - kubectl config set-context default --cluster=k8s --user=admin --namespace=streamtest-ai
    - kubectl config use-context default
    - sed -i "s|gcr.io/streamtest-ai/frontend:latest|$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA|g" k8s/deployment.yaml
    - kubectl apply -f k8s/deployment.yaml
  only:
    - main
```

---

3. Jenkins Pipeline (Jenkinsfile)

Save this file as `Jenkinsfile` in your repository root.

```groovy
pipeline {
    agent any

    environment {
        DOCKER_REGISTRY = 'streamtest'
        IMAGE_NAME = 'frontend'
        CREDENTIALS_ID = 'docker-hub-credentials'
        K8S_CREDENTIALS_ID = 'kubernetes-config-file'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Install Dependencies') {
            steps {
                sh 'npm ci'
            }
        }

        stage('Lint') {
            steps {
                sh 'npm run lint'
            }
```

```
        }

        stage('Run Tests') {
            steps {
                // Run tests inside Docker container helpers or isolated runners
                sh 'npm run test:unit'
                sh 'npx playwright install --with-deps && npm run test:e2e'
            }
        }

        stage('Vulnerability Check') {
            steps {
                sh 'npm audit --audit-level=high || true'
            }
        }

        stage('Build & Push Docker Image') {
            when {
                branch 'main'
            }
            steps {
                script {
                    docker.withRegistry('', CREDENTIALS_ID) {
                        def customImage = docker.build("${DOCKER_REGISTRY}/${IMAGE_NAME}:${env.BUILD_NUMBER}")
                        customImage.push()
                        customImage.push('latest')
                    }
                }
            }
        }

        stage('Deploy to Kubernetes') {
            when {
                branch 'main'
            }
            steps {
                configFileProvider([configFile(fileId: K8S_CREDENTIALS_ID, variable: 'KUBECONFIG')]) {
                    sh 'kubectl --kubeconfig=$KUBECONFIG apply -f k8s/deployment.yaml'
                    sh "kubectl --kubeconfig=$KUBECONFIG set image deployment/web-deployment
web=${DOCKER_REGISTRY}/${IMAGE_NAME}:${env.BUILD_NUMBER} -n streamtest-ai"
                }
            }
        }
    }

    post {
        always {
            cleanWs()
        }
        success {
            echo 'Deployment completed successfully!'
        }
        failure {
            echo 'Pipeline failed. Please inspect build outputs.'
        }
    }
}
```

File: package.json

```
{
  "name": "streamtest-ai",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev --turbopack",
    "build": "next build --turbopack",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "framer-motion": "^12.42.0",
    "lucide-react": "^1.22.0",
    "next": "15.5.19",
    "react": "19.1.0",
    "react-dom": "19.1.0",
    "recharts": "^3.9.0"
  },
  "devDependencies": {
    "@eslint/eslintrc": "^3",
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "15.5.19",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

File: next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  output: "standalone",
  eslint: {
    ignoreDuringBuilds: true,
  },
  typescript: {
    ignoreBuildErrors: true,
  }
};

export default nextConfig;
```

File: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

File: src/app/globals.css

```
@import "tailwindcss";

@theme {
  --color-primary: var(--primary);
  --color-secondary: var(--secondary);
  --color-success: var(--success);
  --color-warning: var(--warning);
  --color-danger: var(--danger);
  --color-border: var(--border);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-sidebar: var(--sidebar);

  --animate-shimmer: shimmer 2.5s infinite linear;
  --animate-fade-in-up: fadeInUp 0.4s ease-out forwards;

  @keyframes shimmer {
    0% { background-position: -200% 0; }
    100% { background-position: 200% 0; }
  }
  @keyframes fadeInUp {
    from {
      opacity: 0;
      transform: translateY(12px);
    }
    to {
      opacity: 1;
      transform: translateY(0);
    }
  }
}

:root {
  /* Default: Dark Mode as requested in the design criteria */
  --background: #0f172a;
  --foreground: #f8fafc;

  --card: rgba(30, 41, 59, 0.7);
  --card-foreground: #f8fafc;
  --sidebar: rgba(15, 23, 42, 0.85);

  --primary: #6366f1;
  --secondary: #8b5cf6;
  --success: #22c55e;
  --warning: #f59e0b;
  --danger: #ef4444;
  --border: rgba(255, 255, 255, 0.08);
}

.light-theme {
  --background: #f1f5f9;
  --foreground: #0f172a;

  --card: rgba(255, 255, 255, 0.8);
  --card-foreground: #1e293b;
  --sidebar: rgba(255, 255, 255, 0.95);

  --primary: #4f46e5;
  --secondary: #7c3aed;
  --success: #16a34a;
  --warning: #d97706;
```

```
  --danger: #dc2626;
  --border: rgba(15, 23, 42, 0.08);
}

/* Custom Scrollbars */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}
::-webkit-scrollbar-track {
  background: transparent;
}
::-webkit-scrollbar-thumb {
  background: rgba(148, 163, 184, 0.3);
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: rgba(148, 163, 184, 0.5);
}

body {
  background-color: var(--background);
  color: var(--foreground);
  transition: background-color 0.3s ease, color 0.3s ease;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, 'Open
Sans', 'Helvetica Neue', sans-serif;
  overflow-x: hidden;
}

/* Premium Glassmorphic Layouts */
.glass-panel {
  background: var(--card);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid var(--border);
  box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.2);
}

.glass-card {
  background: rgba(255, 255, 255, 0.02);
  border: 1px solid rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(4px);
  border-radius: 0.75rem;
  transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
}

.light-theme .glass-card {
  background: rgba(15, 23, 42, 0.02);
  border: 1px solid rgba(15, 23, 42, 0.05);
}

.glass-card:hover {
  transform: translateY(-2px);
  border-color: rgba(99, 102, 241, 0.4);
  box-shadow: 0 10px 20px -10px rgba(99, 102, 241, 0.3);
}

/* Dynamic glow effects */
.glow-primary {
  box-shadow: 0 0 20px -2px rgba(99, 102, 241, 0.45);
}

.glow-secondary {
  box-shadow: 0 0 20px -2px rgba(139, 92, 246, 0.45);
```

```
}

.glow-success {
  box-shadow: 0 0 20px -2px rgba(34, 197, 94, 0.45);
}

/* Custom glowing badge animations */
.glow-pulse {
  position: relative;
}
.glow-pulse::after {
  content: '';
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  width: 100%;
  height: 100%;
  border-radius: 9999px;
  border: 1px solid currentColor;
  animation: ping 1.5s cubic-bezier(0, 0, 0.2, 1) infinite;
  opacity: 0.4;
}

@keyframes ping {
  75%, 100% {
    transform: translate(-50%, -50%) scale(1.6);
    opacity: 0;
  }
}
```

File: src/app/layout.tsx

```
import type { Metadata } from "next";
import "./globals.css";
import LayoutShell from "@/components/layout-shell";

export const metadata: Metadata = {
  title: "StreamTest AI - SaaS Live Streaming Testing Agent",
  description: "Enterprise-grade testing platform for live-streaming SaaS products. Validate security, load, streams,
channels, and billing flows.",
  icons: {
    icon: "/favicon.ico",
  }
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en">
      <body className="antialiased">
        <LayoutShell>{children}</LayoutShell>
      </body>
    </html>
  );
}
```

File: src/components/layout-shell.tsx

```
"use client";

import React, { useState, useEffect } from "react";
import Link from "next/link";
import { usePathname } from "next/navigation";
import {
  LayoutDashboard,
  ShieldCheck,
  Zap,
  Activity,
  Layers,
  FileSpreadsheet,
  Settings,
  Tv,
  KeyRound,
  CreditCard,
  Bell,
  Sun,
  Moon,
  Menu,
  X,
  Bot
} from "lucide-react";

interface LayoutShellProps {
  children: React.ReactNode;
}

export default function LayoutShell({ children }: LayoutShellProps) {
  const pathname = usePathname();
  const [isDarkMode, setIsDarkMode] = useState(true);
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);
  const [notifications, setNotifications] = useState<string[]>([
    "Security scan identified 3 potential vulnerabilities.",
    "FFmpeg worker-03 auto-recovered successfully.",
    "Billing test suite executed: All 12 assertions passed."
  ]);
  const [showNotifications, setShowNotifications] = useState(false);

  useEffect(() => {
    const body = document.body;
    if (isDarkMode) {
      body.classList.remove("light-theme");
    } else {
      body.classList.add("light-theme");
    }
  }, [isDarkMode]);

  const navItems = [
    { name: "Dashboard", href: "/", icon: LayoutDashboard },
    { name: "Authentication Test", href: "/auth-testing", icon: KeyRound },
    { name: "Subscription & Billing", href: "/subscriptions-testing", icon: CreditCard },
    { name: "Channel Integrations", href: "/channels-testing", icon: Tv },
    { name: "Streaming Engine", href: "/streaming-testing", icon: Activity },
    { name: "AI Case Generator", href: "/ai-generator", icon: Bot },
    { name: "Security Scanner", href: "/security-scanner", icon: ShieldCheck },
    { name: "Performance Tester", href: "/load-tester", icon: Zap },
    { name: "Admin Dashboard", href: "/admin-panel", icon: Settings },
    { name: "System Reports", href: "/reports", icon: FileSpreadsheet }
  ];
```

```
  return (
    <div className="min-h-screen flex flex-col md:flex-row bg-[var(--background)] text-[var(--foreground)]
transition-colors duration-300">

      {/* Sidebar - Desktop */}
      <aside className="hidden md:flex flex-col w-64 bg-sidebar backdrop-blur-xl border-r border-[var(--border)]
sticky top-0 h-screen z-20 shrink-0">
        {/* Brand Logo */}
        <div className="h-16 flex items-center px-6 border-b border-[var(--border)] gap-2">
          <div className="w-8 h-8 rounded-lg bg-gradient-to-tr from-primary to-secondary flex items-center
justify-center font-bold text-white text-lg shadow-md glow-primary">
            S
          </div>
          <span className="text-xl font-bold tracking-wider bg-clip-text text-transparent bg-gradient-to-r
from-primary to-secondary">
            StreamTest AI
          </span>
        </div>

        {/* Navigation links */}
        <nav className="flex-1 px-4 py-6 space-y-1.5 overflow-y-auto">
          {navItems.map((item) => {
            const Icon = item.icon;
            const isActive = pathname === item.href;
            return (
              <Link
                key={item.href}
                href={item.href}
                className={`flex items-center px-4 py-3 rounded-xl transition-all duration-200 group text-sm
font-medium ${
                  isActive
                    ? "bg-gradient-to-r from-primary/20 to-secondary/10 border border-primary/30 text-white"
                    : "text-slate-400 hover:bg-white/5 hover:text-white"
                }`}
              >
                <Icon
                  className={`w-5 h-5 mr-3 transition-transform duration-200 group-hover:scale-110 ${
                    isActive ? "text-primary" : "text-slate-400 group-hover:text-white"
                  }`}
                />
                {item.name}
                {item.href === "/streaming-testing" && (
                  <span className="ml-auto w-2 h-2 rounded-full bg-success glow-pulse" />
                )}
              </Link>
            );
          })}
        </nav>

        {/* Footer actions (User, Theme toggle) */}
        <div className="p-4 border-t border-[var(--border)] bg-slate-900/10 space-y-2">
          <div className="flex items-center justify-between">
            <div className="flex items-center gap-3">
              <div className="w-9 h-9 rounded-full bg-gradient-to-tr from-primary to-secondary flex items-center
justify-center font-bold text-white text-xs">
                JD
              </div>
              <div>
                <p className="text-xs font-semibold text-white">John Dev</p>
                <p className="text-[10px] text-slate-400">Enterprise QA</p>
              </div>
            </div>

            <button
```

```
              onClick={() => setIsDarkMode(!isDarkMode)}
              className="p-2 rounded-lg hover:bg-white/10 text-slate-400 hover:text-white transition-colors"
              title="Toggle Theme"
            >
              {isDarkMode ? <Sun className="w-4 h-4 text-warning" /> : <Moon className="w-4 h-4 text-primary" />}
            </button>
          </div>
        </div>
      </aside>

      {/* Mobile Header */}
      <header className="flex md:hidden items-center justify-between px-6 h-16 bg-sidebar/90 backdrop-blur-md border-b
border-[var(--border)] sticky top-0 z-30 w-full">
        <div className="flex items-center gap-2">
          <div className="w-8 h-8 rounded-lg bg-gradient-to-tr from-primary to-secondary flex items-center
justify-center font-bold text-white text-md">
            S
          </div>
          <span className="text-lg font-bold tracking-wider bg-clip-text text-transparent bg-gradient-to-r
from-primary to-secondary">
            StreamTest AI
          </span>
        </div>

        <div className="flex items-center gap-3">
          <button
            onClick={() => setIsDarkMode(!isDarkMode)}
            className="p-2 rounded-lg hover:bg-white/10 text-slate-400 hover:text-white"
          >
            {isDarkMode ? <Sun className="w-4 h-4 text-warning" /> : <Moon className="w-4 h-4 text-primary" />}
          </button>
          <button
            onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
            className="p-2 rounded-lg hover:bg-white/10 text-slate-400 hover:text-white"
          >
            {isMobileMenuOpen ? <X className="w-6 h-6" /> : <Menu className="w-6 h-6" />}
          </button>
        </div>
      </header>

      {/* Mobile Navigation Drawer */}
      {isMobileMenuOpen && (
        <div className="md:hidden fixed inset-0 z-40 bg-black/50 backdrop-blur-sm" onClick={() =>
setIsMobileMenuOpen(false)}>
          <aside className="w-64 max-w-xs h-full bg-sidebar flex flex-col border-r border-[var(--border)]"
onClick={(e) => e.stopPropagation()}>
            <div className="h-16 flex items-center px-6 border-b border-[var(--border)] gap-2">
              <div className="w-8 h-8 rounded-lg bg-gradient-to-tr from-primary to-secondary flex items-center
justify-center font-bold text-white text-md">
                S
              </div>
              <span className="text-md font-bold tracking-wider text-white">StreamTest AI</span>
            </div>

            <nav className="flex-1 px-4 py-4 space-y-1 overflow-y-auto">
              {navItems.map((item) => {
                const Icon = item.icon;
                const isActive = pathname === item.href;
                return (
                  <Link
                    key={item.href}
                    href={item.href}
                    onClick={() => setIsMobileMenuOpen(false)}
                    className={`flex items-center px-4 py-2.5 rounded-xl transition-all duration-200 text-sm
```

```
font-medium ${
                      isActive
                        ? "bg-primary/20 border border-primary/30 text-white"
                        : "text-slate-400 hover:bg-white/5 hover:text-white"
                    }`}
                  >
                    <Icon className="w-5 h-5 mr-3 text-slate-400" />
                    {item.name}
                  </Link>
                );
              })}
            </nav>
          </aside>
        </div>
      )}

      {/* Main Content Area */}
      <div className="flex-1 flex flex-col overflow-x-hidden min-h-0">
        {/* Top Header - Desktop only */}
        <header className="hidden md:flex items-center justify-between px-8 h-16 border-b border-[var(--border)]
bg-slate-900/5 backdrop-blur-md sticky top-0 z-10">
          <div className="text-sm text-slate-400">
            Platform Status: <span className="text-success font-semibold">Active & Healthy</span>
          </div>

          <div className="flex items-center gap-6">
            {/* Notification Drawer */}
            <div className="relative">
              <button
                onClick={() => setShowNotifications(!showNotifications)}
                className="p-2 rounded-xl hover:bg-white/5 text-slate-400 hover:text-white transition-colors relative"
              >
                <Bell className="w-5 h-5" />
                {notifications.length > 0 && (
                  <span className="absolute top-1.5 right-1.5 w-2 h-2 rounded-full bg-danger glow-pulse" />
                )}
              </button>

              {showNotifications && (
                <div className="absolute right-0 mt-3 w-80 glass-panel border border-[var(--border)] rounded-2xl
overflow-hidden shadow-2xl z-50">
                  <div className="p-4 border-b border-[var(--border)] bg-slate-950/20 flex justify-between
items-center">
                    <h3 className="font-semibold text-sm">Notifications</h3>
                    <button
                      onClick={() => setNotifications([])}
                      className="text-xs text-primary hover:underline"
                    >
                      Clear all
                    </button>
                  </div>
                  <div className="max-h-72 overflow-y-auto">
                    {notifications.length === 0 ? (
                      <div className="p-4 text-center text-xs text-slate-500">No new alerts</div>
                    ) : (
                      notifications.map((note, index) => (
                        <div key={index} className="p-3.5 border-b border-[var(--border)/5] hover:bg-white/2 text-xs
leading-relaxed text-slate-300">
                          {note}
                        </div>
                      ))
                    )}
                  </div>
                </div>
```

```
              )}
            </div>

            <div className="h-6 w-[1px] bg-[var(--border)]" />

            <div className="flex items-center gap-2">
              <div className="w-2.5 h-2.5 rounded-full bg-success glow-pulse mr-1" />
              <span className="text-xs font-semibold text-slate-300">RTMP/HLS Sandbox: Online</span>
            </div>
          </div>
        </header>

        {/* Page Children Render */}
        <main className="flex-1 p-6 md:p-8 overflow-y-auto max-w-7xl w-full mx-auto">
          {children}
        </main>
      </div>
    </div>
  );
}
```

File: src/app/page.tsx

```
"use client";

import React, { useEffect, useState } from "react";
import Link from "next/link";
import {
  Activity,
  Tv,
  Cpu,
  Database,
  Layers,
  ArrowRight,
  TrendingUp,
  AlertTriangle,
  Play,
  StopCircle,
  RefreshCw,
  HardDrive
} from "lucide-react";
import {
  AreaChart,
  Area,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  ResponsiveContainer,
  BarChart,
  Bar,
  Cell
} from "recharts";

interface Stream {
  id: string;
  title: string;
  video: string;
  channels: string[];
  scheduledTime: string;
  status: string;
  fps: number;
  bitrate: number;
  resolution: string;
  uptime: string;
  frameDrops: number;
  networkStatus: string;
  retryCount: number;
}

export default function Dashboard() {
  const [mounted, setMounted] = useState(false);
  const [streams, setStreams] = useState<Stream[]>([]);
  const [logs, setLogs] = useState<string[]>([]);
  const [system, setSystem] = useState({
    cpuUsage: 42.5,
    ramUsage: 58.2,
    storageUsage: 31.8,
    redisQueue: 0,
    activeWorkers: 4,
    ffmpegProcesses: 2
  });
  const [isRefreshing, setIsRefreshing] = useState(false);
```

```
// CPU history for charts
const [cpuHistory, setCpuHistory] = useState([
  { name: "10m", cpu: 38, ram: 55 },
  { name: "8m", cpu: 45, ram: 56 },
  { name: "6m", cpu: 41, ram: 57 },
  { name: "4m", cpu: 52, ram: 58 },
  { name: "2m", cpu: 40, ram: 58 },
  { name: "Now", cpu: 42, ram: 58 }
]);

const fetchState = async () => {
  setIsRefreshing(true);
  try {
    const res = await fetch("/api/stream-engine");
    const data = await res.json();
    setStreams(data.streams);
    setLogs(data.logs);
    setSystem(data.system);

    // Append current CPU to history
    setCpuHistory(prev => {
      const next = [...prev.slice(1)];
      next.push({
        name: "Now",
        cpu: Math.round(data.system.cpuUsage),
        ram: Math.round(data.system.ramUsage)
      });
      return next;
    });
  } catch (e) {
    console.error("Error fetching state:", e);
  } finally {
    setIsRefreshing(false);
  }
};

useEffect(() => {
  setMounted(true);
  fetchState();

  // Poll metrics every 6 seconds
  const interval = setInterval(() => {
    fetchState();
  }, 6000);

  return () => clearInterval(interval);
}, []);

const triggerAction = async (action: string, streamId?: string) => {
  try {
    await fetch("/api/stream-engine", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ action, streamId })
    });
    fetchState();
  } catch (e) {
    console.error(e);
  }
};

if (!mounted) return null;

return (
```

```
    <div className="space-y-8 animate-fade-in-up">
      {/* Welcome banner */}
      <div className="flex flex-col md:flex-row md:items-center md:justify-between gap-4">
        <div>
          <h1 className="text-3xl font-extrabold text-white tracking-tight">StreamTest AI Dashboard</h1>
          <p className="text-slate-400 mt-1">Real-time status monitoring, streaming nodes health, and testing
reports.</p>
        </div>
        <div className="flex items-center gap-3">
          <button
            onClick={() => triggerAction("reset")}
            className="px-4 py-2 rounded-xl text-xs font-semibold bg-white/5 border border-[var(--border)]
hover:bg-white/10 text-slate-300 hover:text-white transition-colors"
          >
            Reset Simulator
          </button>
          <button
            onClick={fetchState}
            disabled={isRefreshing}
            className="flex items-center px-4 py-2 rounded-xl text-xs font-semibold bg-primary hover:bg-primary-hover
text-white transition-colors gap-2 shadow-lg glow-primary"
          >
            <RefreshCw className={`w-3.5 h-3.5 ${isRefreshing ? "animate-spin" : ""}`} />
            {isRefreshing ? "Syncing..." : "Refresh Status"}
          </button>
        </div>
      </div>

      {/* KPI Stats Grid */}
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6">
        <div className="glass-panel p-6 rounded-2xl relative overflow-hidden">
          <div className="absolute top-0 right-0 p-3 opacity-10">
            <Activity className="w-16 h-16 text-primary" />
          </div>
          <p className="text-xs font-medium text-slate-400 uppercase tracking-wider">Active Streams</p>
          <p className="text-3xl font-bold text-white mt-2">
            {streams.filter(s => s.status === "streaming").length}
          </p>
          <div className="flex items-center gap-1.5 text-xs text-success mt-3 font-medium">
            <span className="w-2 h-2 rounded-full bg-success glow-pulse" />
            Live Sandbox Connected
          </div>
        </div>

        <div className="glass-panel p-6 rounded-2xl relative overflow-hidden">
          <div className="absolute top-0 right-0 p-3 opacity-10">
            <Tv className="w-16 h-16 text-secondary" />
          </div>
          <p className="text-xs font-medium text-slate-400 uppercase tracking-wider">Connected Channels</p>
          <p className="text-3xl font-bold text-white mt-2">2 <span className="text-sm font-normal text-slate-400">/ 6
max</span></p>
          <p className="text-xs text-slate-400 mt-3 flex items-center gap-1">
            <TrendingUp className="w-3.5 h-3.5 text-secondary" />
            YouTube, Facebook linked
          </p>
        </div>

        <div className="glass-panel p-6 rounded-2xl relative overflow-hidden">
          <div className="absolute top-0 right-0 p-3 opacity-10">
            <Layers className="w-16 h-16 text-success" />
          </div>
          <p className="text-xs font-medium text-slate-400 uppercase tracking-wider">Queue Workers</p>
          <p className="text-3xl font-bold text-white mt-2">
            {system.activeWorkers} <span className="text-sm font-normal text-slate-400">online</span>
```

```
          </p>
          <p className="text-xs text-success mt-3 font-medium">
            BullMQ tasks: {system.redisQueue} pending
          </p>
        </div>

        <div className="glass-panel p-6 rounded-2xl relative overflow-hidden">
          <div className="absolute top-0 right-0 p-3 opacity-10">
            <HardDrive className="w-16 h-16 text-warning" />
          </div>
          <p className="text-xs font-medium text-slate-400 uppercase tracking-wider">Storage Usage</p>
          <p className="text-3xl font-bold text-white mt-2">
            {system.storageUsage.toFixed(1)}%
          </p>
          <p className="text-xs text-slate-400 mt-3 font-medium">
            1.59 GB of 5.00 GB trial limit
          </p>
        </div>
      </div>

      {/* Main Grid: Streaming Sandbox & System resource chart */}
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {/* Left Column: Streams list (2/3 width) */}
        <div className="lg:col-span-2 space-y-6">
          <div className="glass-panel p-6 rounded-2xl">
            <div className="flex items-center justify-between border-b border-[var(--border)] pb-4 mb-6">
              <h2 className="text-lg font-bold text-white flex items-center gap-2">
                <Activity className="w-5 h-5 text-primary" />
                Active Stream Validations
              </h2>
              <span className="text-xs text-slate-400 font-semibold">Sandbox Environment</span>
            </div>

            <div className="space-y-4">
              {streams.length === 0 ? (
                <p className="text-slate-400 text-sm text-center py-6">No streams currently active.</p>
              ) : (
                streams.map(stream => (
                  <div key={stream.id} className="p-5 rounded-xl bg-slate-900/40 border border-[var(--border)]
hover:border-slate-700 transition-colors space-y-4">
                    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-2">
                      <div>
                        <h3 className="font-semibold text-white text-base">{stream.title}</h3>
                        <p className="text-xs text-slate-400 mt-0.5">Asset: {stream.video} | ID: {stream.id}</p>
                      </div>

                      {/* Badge status */}
                      <span className={`px-3 py-1 rounded-full text-xs font-medium w-fit flex items-center gap-1.5 ${
                        stream.status === "streaming"
                          ? "bg-success/10 text-success border border-success/20"
                          : "bg-danger/10 text-danger border border-danger/20 glow-pulse"
                      }`}>
                        <span className={`w-1.5 h-1.5 rounded-full ${stream.status === "streaming" ? "bg-success
animate-pulse" : "bg-danger"}`} />
                        {stream.status.toUpperCase()}
                      </span>
                    </div>

                    <div className="grid grid-cols-2 sm:grid-cols-4 gap-4 py-2 border-y border-[var(--border)/5]
text-xs">
                      <div>
                        <p className="text-slate-400">Resolution / FPS</p>
                        <p className="font-semibold text-white mt-1">{stream.resolution} @ {stream.fps}fps</p>
```

```
                    </div>
                    <div>
                      <p className="text-slate-400">Bitrate</p>
                      <p className="font-semibold text-white mt-1">{(stream.bitrate / 1000).toFixed(2)} Mbps</p>
                    </div>
                    <div>
                      <p className="text-slate-400">Uptime</p>
                      <p className="font-semibold text-white mt-1">{stream.uptime}</p>
                    </div>
                    <div>
                      <p className="text-slate-400">Dropped Frames</p>
                      <p className={`font-semibold mt-1 ${stream.frameDrops > 10 ? "text-warning" : "text-white"}`}>
                        {stream.frameDrops} frames
                      </p>
                    </div>
                  </div>

                  {/* Actions panel */}
                  <div className="flex flex-wrap items-center justify-between gap-3 pt-1">
                    <div className="flex items-center gap-2 text-[11px] text-slate-400">
                      <span>Outputs:</span>
                      {stream.channels.map(ch => (
                        <span key={ch} className="px-2 py-0.5 rounded bg-white/5 border border-[var(--border)]
text-white text-[10px]">
                          {ch}
                        </span>
                      ))}
                      {stream.retryCount > 0 && (
                        <span className="text-warning font-semibold ml-2">
                          ({stream.retryCount} auto-recoveries)
                        </span>
                      )}
                    </div>

                    <div className="flex items-center gap-2">
                      {stream.status === "streaming" ? (
                        <button
                          onClick={() => triggerAction("crash_worker", stream.id)}
                          className="flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-semibold
bg-danger/10 hover:bg-danger/20 text-danger border border-danger/20 transition-colors"
                        >
                          <AlertTriangle className="w-3.5 h-3.5" />
                          Simulate Node Crash
                        </button>
                      ) : (
                        <button
                          onClick={() => triggerAction("recover_worker", stream.id)}
                          className="flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-semibold
bg-success/10 hover:bg-success/20 text-success border border-success/20 transition-colors"
                        >
                          <Play className="w-3.5 h-3.5" />
                          Trigger Auto Recovery
                        </button>
                      )}
                    </div>
                  </div>
                </div>
              ))
            )}
          </div>
        </div>

        {/* Infrastructure Metrics charts */}
        <div className="glass-panel p-6 rounded-2xl">
```

```
    <h2 className="text-lg font-bold text-white mb-6 flex items-center gap-2">
      <Cpu className="w-5 h-5 text-secondary" />
      Infrastructure Resource Monitoring
    </h2>

    <div className="h-64 w-full">
      <ResponsiveContainer width="100%" height="100%">
        <AreaChart
          data={cpuHistory}
          margin={{ top: 10, right: 10, left: -20, bottom: 0 }}
        >
          <defs>
            <linearGradient id="colorCpu" x1="0" y1="0" x2="0" y2="1">
              <stop offset="5%" stopColor="#6366f1" stopOpacity={0.4}/>
              <stop offset="95%" stopColor="#6366f1" stopOpacity={0}/>
            </linearGradient>
            <linearGradient id="colorRam" x1="0" y1="0" x2="0" y2="1">
              <stop offset="5%" stopColor="#8b5cf6" stopOpacity={0.4}/>
              <stop offset="95%" stopColor="#8b5cf6" stopOpacity={0}/>
            </linearGradient>
          </defs>
          <CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.05)" />
          <XAxis dataKey="name" stroke="#94a3b8" fontSize={11} />
          <YAxis stroke="#94a3b8" fontSize={11} domain={[0, 100]} />
          <Tooltip
            contentStyle={{
              backgroundColor: "#1e293b",
              borderColor: "rgba(255,255,255,0.08)",
              borderRadius: "12px",
              color: "#f8fafc"
            }}
          />
          <Area
            type="monotone"
            dataKey="cpu"
            name="CPU Usage %"
            stroke="#6366f1"
            fillOpacity={1}
            fill="url(#colorCpu)"
          />
          <Area
            type="monotone"
            dataKey="ram"
            name="RAM Usage %"
            stroke="#8b5cf6"
            fillOpacity={1}
            fill="url(#colorRam)"
          />
        </AreaChart>
      </ResponsiveContainer>
    </div>
    <div className="flex items-center justify-center gap-6 mt-4 text-xs">
      <div className="flex items-center gap-1.5 text-primary">
        <span className="w-2.5 h-2.5 rounded-sm bg-primary" />
        FFmpeg Pods (CPU)
      </div>
      <div className="flex items-center gap-1.5 text-secondary">
        <span className="w-2.5 h-2.5 rounded-sm bg-secondary" />
        Redis & Core Service (RAM)
      </div>
    </div>
  </div>
</div>
```

```
        {/* Right Column: Engine Logs & Quick Links (1/3 width) */}
        <div className="space-y-6">
          {/* Logs panel */}
          <div className="glass-panel p-6 rounded-2xl flex flex-col h-[400px]">
            <h2 className="text-lg font-bold text-white mb-4 flex items-center gap-2">
              <Database className="w-5 h-5 text-warning" />
              Stream Logs Feed
            </h2>
            <div className="flex-1 overflow-y-auto bg-slate-950/50 rounded-xl p-4 font-mono text-[10px] space-y-2
border border-[var(--border)] leading-relaxed scrollbar-thin">
              {logs.map((log, index) => (
                <div key={index} className={`whitespace-pre-wrap ${
                  log.includes("[CRITICAL]")
                    ? "text-danger"
                    : log.includes("[WARN]")
                      ? "text-warning"
                      : log.includes("[RECOVERY]")
                        ? "text-success font-semibold"
                        : "text-slate-300"
                }`}>
                  {log}
                </div>
              ))}
            </div>
          </div>

          {/* Quick Nav Tools */}
          <div className="glass-panel p-6 rounded-2xl space-y-4">
            <h2 className="text-base font-bold text-white">Agent Validation Shortcuts</h2>
            <div className="grid grid-cols-1 gap-2.5">
              <Link
                href="/auth-testing"
                className="flex items-center justify-between p-3.5 rounded-xl bg-white/2 hover:bg-white/5 border
border-[var(--border)] text-xs transition-all duration-200 text-slate-300 hover:text-white"
              >
                <span>Authentication Workflows</span>
                <ArrowRight className="w-4 h-4 text-primary" />
              </Link>
              <Link
                href="/security-scanner"
                className="flex items-center justify-between p-3.5 rounded-xl bg-white/2 hover:bg-white/5 border
border-[var(--border)] text-xs transition-all duration-200 text-slate-300 hover:text-white"
              >
                <span>OWASP Security Scanner</span>
                <ArrowRight className="w-4 h-4 text-success" />
              </Link>
              <Link
                href="/load-tester"
                className="flex items-center justify-between p-3.5 rounded-xl bg-white/2 hover:bg-white/5 border
border-[var(--border)] text-xs transition-all duration-200 text-slate-300 hover:text-white"
              >
                <span>Performance Load Testing</span>
                <ArrowRight className="w-4 h-4 text-warning" />
              </Link>
            </div>
          </div>
        </div>

      </div>
    </div>
  );
}
```

File: src/app/auth-testing/page.tsx

```
"use client";

import React, { useState } from "react";
import { KeyRound, ShieldAlert, CheckCircle, XCircle, Play, RefreshCw, Terminal, Plus } from "lucide-react";

interface TestCase {
  id: string;
  name: string;
  category: "Signup" | "Login" | "Google OAuth" | "GitHub OAuth" | "Forgot Password" | "Email Verification" | "Phone
OTP";
  assertionsCount: number;
  status: "idle" | "running" | "passed" | "failed";
  playwrightLogs: string[];
  assertions: string[];
}

export default function AuthTesting() {
  const [testCases, setTestCases] = useState<TestCase[]>([
    {
      id: "TC-AUTH-S1",
      name: "User Sign Up & Form Validation Tests",
      category: "Signup",
      assertionsCount: 6,
      status: "idle",
      assertions: [
        "Assert validation error on invalid email formatting",
        "Assert warning message when email is already taken",
        "Assert strength meter block on short/weak passwords",
        "Assert database persistence for valid registrations",
        "Assert verification email dispatch payload matches schema",
        "Assert redirection to onboarding page post-registration"
      ],
      playwrightLogs: [
        "PLAYWRIGHT: Navigating to https://streamtest-saas.example.com/signup",
        "PLAYWRIGHT: Fuzzing email input with values: '', 'plainaddress', '@missing-domain.com'",
        "PLAYWRIGHT: Asserting validation messages are visible",
        "PLAYWRIGHT: Inserting existing user email 'existing_user@streamtest.ai'",
        "PLAYWRIGHT: Asserting server returns HTTP 409 Conflict",
        "PLAYWRIGHT: Typing password '123' and asserting 'Password too weak' banner",
        "PLAYWRIGHT: Completing valid signup form and verifying HTTP 201 Created response",
        "TEST: Verify email webhook capture and token structure matches database",
        "RESULT: 6/6 Assertions Passed. Test Passed."
      ]
    },
    {
      id: "TC-AUTH-L1",
      name: "Sign In Security & Lockout Tests",
      category: "Login",
      assertionsCount: 5,
      status: "idle",
      assertions: [
        "Assert login success with valid credentials & JWT generation",
        "Assert authorization error with incorrect password",
        "Assert lockout activation after exactly 5 failed login attempts",
        "Assert session cookie matches security flags (HttpOnly, Secure, SameSite)",
        "Assert session termination on logout command"
      ],
      playwrightLogs: [
        "PLAYWRIGHT: Navigating to https://streamtest-saas.example.com/login",
        "PLAYWRIGHT: Typing valid credentials and clicking Submit",
        "PLAYWRIGHT: Asserting JWT Cookie exists and is configured HttpOnly/Secure",
```

```
      "PLAYWRIGHT: Running brute-force simulation (5 invalid attempts with same email)",
      "PLAYWRIGHT: Asserting login endpoint responds with HTTP 423 Locked Out",
      "PLAYWRIGHT: Verification of account lockout release timer triggers in DB",
      "RESULT: 5/5 Assertions Passed. Test Passed."
    ]
  },
  {
    id: "TC-AUTH-G1",
    name: "Google OAuth 2.0 Integration Tests",
    category: "Google OAuth",
    assertionsCount: 4,
    status: "idle",
    assertions: [
      "Assert callback handles successful authentication correctly",
      "Assert login failure handles cancelled consent accurately",
      "Assert JWT payload parses user profile (email, name, picture)",
      "Assert automated access token refresh cycle works"
    ],
    playwrightLogs: [
      "PLAYWRIGHT: Simulating click on 'Sign in with Google' button",
      "PLAYWRIGHT: Intercepting OAuth callback redirect: code=4/0AX4X...&state=secure_state",
      "PLAYWRIGHT: Mocking Google Identity Platform exchange endpoint returning profile data",
      "PLAYWRIGHT: Asserting application database creates record for new user",
      "PLAYWRIGHT: Verifying application JWT session creation",
      "RESULT: 4/4 Assertions Passed. Test Passed."
    ]
  },
  {
    id: "TC-AUTH-GH1",
    name: "GitHub OAuth Integration Tests",
    category: "GitHub OAuth",
    assertionsCount: 4,
    status: "idle",
    assertions: [
      "Assert callback exchanges auth code for access token successfully",
      "Assert state parameter is verified against CSRF attacks",
      "Assert email is fetched from GitHub secondary private emails API",
      "Assert user profile data maps correctly to user record in database"
    ],
    playwrightLogs: [
      "PLAYWRIGHT: Simulating click on 'Sign in with GitHub' button",
      "PLAYWRIGHT: Intercepting callback redirect: code=git_code_8831&state=github_csrf_state",
      "PLAYWRIGHT: Exchanging auth code for access token at github.com/login/oauth/access_token",
      "PLAYWRIGHT: Fetching private emails from api.github.com/user/emails - Success",
      "TEST: Verifying state parameter match - Passed",
      "DATABASE: Merging GitHub profile (Mohith831732108820440699) with users registry",
      "RESULT: 4/4 Assertions Passed. Test Passed."
    ]
  },
  {
    id: "TC-AUTH-OTP1",
    name: "Phone Number OTP SMS Verification Tests",
    category: "Phone OTP",
    assertionsCount: 5,
    status: "idle",
    assertions: [
      "Assert SMS payload complies with Twilio gateway schemas",
      "Assert secure 6-digit OTP code is generated and stored in Redis cache",
      "Assert OTP rate limiting triggers on exceeding 3 resend commands",
      "Assert verification state sets to active on correct 6-digit match",
      "Assert verification blocks expired OTP codes (after 120s limit)"
    ],
    playwrightLogs: [
      "PLAYWRIGHT: Entering phone number '+91 8317321088' and clicking Request OTP",
```

```
      "TEST: Verify Twilio SMS API webhook trigger - Success",
      "REDIS: Set temporary OTP code key with TTL 120s: val='482910'",
      "PLAYWRIGHT: Typing incorrect OTP code '111111' and asserting error modal",
      "PLAYWRIGHT: Triggering resend SMS 3 times and verifying 429 rate limit blocker",
      "PLAYWRIGHT: Typing correct OTP code '482910' and checking DB verification state",
      "RESULT: 5/5 Assertions Passed. Test Passed."
    ]
  },
  {
    id: "TC-AUTH-P1",
    name: "Forgot Password Recovery Boundary Tests",
    category: "Forgot Password",
    assertionsCount: 4,
    status: "idle",
    assertions: [
      "Assert reset link mail payload contains valid cryptographically signed token",
      "Assert error message on expired password reset link (after 1 hour)",
      "Assert invalid / tampered token results in HTTP 400 Bad Request",
      "Assert database password hash changes successfully after recovery completion"
    ],
    playwrightLogs: [
      "PLAYWRIGHT: Navigating to /forgot-password",
      "PLAYWRIGHT: Triggering password reset request for 'qa_tester@streamtest.ai'",
      "TEST: Inspecting mock SMTP server and retrieving signed token: 'pwd_tok_102938'",
      "PLAYWRIGHT: Fuzzing password reset callback page with expired token",
      "PLAYWRIGHT: Asserting 'Link expired' modal displays correctly",
      "PLAYWRIGHT: Completing reset flow with valid token and logging in with new credentials",
      "RESULT: 4/4 Assertions Passed. Test Passed."
    ]
  }
]);

const [activeLogs, setActiveLogs] = useState<string[]>([]);
const [activeTestName, setActiveTestName] = useState<string>("");
const [runningAll, setRunningAll] = useState(false);

const runTestCase = async (index: number) => {
  // Set to running
  setTestCases(prev => {
    const next = [...prev];
    next[index].status = "running";
    return next;
  });

  setActiveTestName(testCases[index].name);
  setActiveLogs([`Starting test case: ${testCases[index].name}...`]);

  const logs = testCases[index].playwrightLogs;
  for (let i = 0; i < logs.length; i++) {
    await new Promise(resolve => setTimeout(resolve, 400));
    setActiveLogs(prev => [...prev, logs[i]]);
  }

  setTestCases(prev => {
    const next = [...prev];
    next[index].status = "passed";
    return next;
  });
};

const runAllTests = async () => {
  setRunningAll(true);
  // Reset all states
  setTestCases(prev => prev.map(t => ({ ...t, status: "idle" })));
```

```
    for (let i = 0; i < testCases.length; i++) {
      await runTestCase(i);
      await new Promise(resolve => setTimeout(resolve, 600));
    }
    setRunningAll(false);
  };

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
            <KeyRound className="w-8 h-8 text-primary" />
            Authentication Testing Module
          </h1>
          <p className="text-slate-400 mt-1 font-medium">Verify credentials validation, security locks, email hooks,
and OAuth callbacks.</p>
        </div>
        <button
          onClick={runAllTests}
          disabled={runningAll}
          className="flex items-center justify-center px-5 py-2.5 rounded-xl font-bold bg-gradient-to-r from-primary
to-secondary hover:from-primary-hover hover:to-secondary-hover text-white transition-all shadow-lg glow-primary gap-2
text-sm"
        >
          {runningAll ? (
            <>
              <RefreshCw className="w-4 h-4 animate-spin" />
              Running Suite...
            </>
          ) : (
            <>
              <Play className="w-4 h-4" />
              Run Full Auth Suite
            </>
          )}
        </button>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
        {/* Left Column: Test Cases (2/3 width) */}
        <div className="lg:col-span-2 space-y-6">
          {testCases.map((tc, index) => (
            <div key={tc.id} className="glass-panel p-6 rounded-2xl space-y-4">
              <div className="flex items-start justify-between gap-4">
                <div>
                  <div className="flex items-center gap-3.5">
                    <span className="text-[10px] uppercase font-bold tracking-wider px-2 py-0.5 rounded bg-white/5
border border-slate-700 text-slate-300">
                      {tc.category}
                    </span>
                    <span className="text-[10px] text-slate-500 font-mono">{tc.id}</span>
                  </div>
                  <h2 className="text-lg font-bold text-white mt-1.5">{tc.name}</h2>
                </div>

                <div className="flex items-center gap-3">
                  <span className={`px-2.5 py-0.5 rounded-full text-xs font-semibold ${
                    tc.status === "passed"
                      ? "bg-success/10 text-success border border-success/20"
                      : tc.status === "failed"
                        ? "bg-danger/10 text-danger border border-danger/20"
```

```
                      : tc.status === "running"
                        ? "bg-primary/10 text-primary border border-primary/20"
                        : "bg-white/5 text-slate-400 border border-[var(--border)]"
                  }`}>
                    {tc.status.toUpperCase()}
                  </span>

                  <button
                    onClick={() => runTestCase(index)}
                    disabled={tc.status === "running" || runningAll}
                    className="p-2 rounded-lg bg-white/5 border border-[var(--border)] hover:bg-primary/20
hover:border-primary hover:text-white text-slate-300 transition-all"
                    title="Run individual test"
                  >
                    <Play className="w-3.5 h-3.5" />
                  </button>
                </div>
              </div>

              {/* Assertions list */}
              <div className="bg-slate-950/20 rounded-xl p-4 border border-[var(--border)/5]">
                <h3 className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-2.5">E2E Playwright
Assertions</h3>
                <ul className="space-y-2 text-xs">
                  {tc.assertions.map((assert, i) => (
                    <li key={i} className="flex items-start gap-2.5 text-slate-300 leading-normal">
                      {tc.status === "passed" ? (
                        <CheckCircle className="w-4 h-4 text-success shrink-0 mt-0.5" />
                      ) : tc.status === "failed" ? (
                        <XCircle className="w-4 h-4 text-danger shrink-0 mt-0.5" />
                      ) : (
                        <span className="w-1.5 h-1.5 rounded-full bg-slate-600 shrink-0 mt-2 ml-1" />
                      )}
                      <span>{assert}</span>
                    </li>
                  ))}
                </ul>
              </div>
            </div>
          ))}

          {/* Future Section placeholder */}
          <div className="glass-panel p-6 rounded-2xl border-dashed flex items-center justify-between text-slate-400">
            <div className="flex items-center gap-3">
              <ShieldAlert className="w-5 h-5 text-slate-500" />
              <div>
                <p className="text-sm font-semibold text-slate-300">Future Security Integrations</p>
                <p className="text-xs text-slate-500">2FA Authenticator & WebAuthn/Passkeys test runners.</p>
              </div>
            </div>
            <button className="flex items-center gap-1 text-xs px-3 py-1.5 rounded-lg border border-[var(--border)]
hover:bg-white/5 transition-colors">
              <Plus className="w-3.5 h-3.5" /> Configure
            </button>
          </div>
        </div>

        {/* Right Column: Execution Log Console (1/3 width) */}
        <div className="space-y-6">
          <div className="glass-panel p-6 rounded-2xl sticky top-24 flex flex-col h-[500px]">
            <div className="flex items-center justify-between border-b border-[var(--border)] pb-4 mb-4">
              <h2 className="text-base font-bold text-white flex items-center gap-2">
                <Terminal className="w-4 h-4 text-primary" />
                Live Assertions terminal
```

```
              </h2>
              <span className="text-[10px] font-mono text-slate-500">Playwright 1.40</span>
            </div>

            <div className="flex-1 overflow-y-auto bg-slate-950/80 rounded-xl p-4 font-mono text-[10px] space-y-2
text-slate-300 border border-slate-800 leading-relaxed scrollbar-thin">
              {activeLogs.length === 0 ? (
                <div className="text-slate-500 text-center py-20">
                  <p>Click "Run" to stream E2E test logs...</p>
                </div>
              ) : (
                <>
                  <div className="text-primary font-semibold border-b border-white/5 pb-1 mb-2">
                    Executing: {activeTestName}
                  </div>
                  {activeLogs.map((log, index) => (
                    <div key={index} className={
                      log.startsWith("RESULT:")
                        ? "text-success font-semibold mt-1 border-t border-white/5 pt-1"
                        : log.startsWith("TEST:")
                          ? "text-secondary"
                          : "text-slate-300"
                    }>
                      {log}
                    </div>
                  ))}
                </>
              )}
            </div>

            {activeLogs.length > 0 && (
              <button
                onClick={() => {
                  setActiveLogs([]);
                  setActiveTestName("");
                }}
                className="mt-4 w-full py-2 bg-white/5 hover:bg-white/10 text-xs font-semibold text-slate-300
hover:text-white rounded-xl border border-[var(--border)] transition-colors"
              >
                Clear Terminal
              </button>
            )}
          </div>
        </div>
      </div>
    </div>
  );
}
```

File: src/app/subscriptions-testing/page.tsx

```
"use client";

import React, { useState } from "react";
import { CreditCard, Check, ShieldCheck, Play, RefreshCw, Layers, DollarSign, Database } from "lucide-react";

interface BillingScenario {
  id: string;
  name: string;
  gateway: "Stripe" | "Razorpay" | "PayPal";
  plan: "Free Trial" | "Standard" | "Premium" | "Pro";
  action: "Upgrade" | "Downgrade" | "Refund" | "Renewal" | "Expiration";
  status: "idle" | "running" | "passed" | "failed";
  assertions: string[];
  logs: string[];
}

export default function SubscriptionsTesting() {
  const [scenarios, setScenarios] = useState<BillingScenario[]>([
    {
      id: "SC-BILL-01",
      name: "Stripe Standard-to-Premium Upgrade",
      gateway: "Stripe",
      plan: "Premium",
      action: "Upgrade",
      status: "idle",
      assertions: [
        "Assert customer.subscription.updated webhook is processed successfully",
        "Assert channel connection limits update from 1 to 3 in DB",
        "Assert storage capacity limit updates from 5GB to 50GB in subscription meta",
        "Assert prorated payment billing record is inserted under billing_history"
      ],
      logs: [
        "STRIPE MOCK: Dispatching webhook event 'customer.subscription.updated'...",
        "WEBHOOK: Received event id 'evt_1O29x8321' from Stripe signature validator",
        "DATABASE: Querying user_id: 'usr_90123' subscription state",
        "DATABASE: Executing UPDATE subscriptions SET plan = 'premium', channel_limit = 3, storage_limit_gb = 50.00",
        "ASSERT: Asserting updated row count equals 1 - Success",
        "DATABASE: Inserting record into billing_history: amount = $29.00, tax = $2.32",
        "RESULT: Subscription upgrade verified. 4/4 Assertions Passed."
      ]
    },
    {
      id: "SC-BILL-02",
      name: "Razorpay Free Trial Expiration Lock",
      gateway: "Razorpay",
      plan: "Free Trial",
      action: "Expiration",
      status: "idle",
      assertions: [
        "Assert scheduler identifies expired subscription records",
        "Assert subscription status is set to 'canceled' in DB",
        "Assert user is blocked from starting a live stream (returns HTTP 403)",
        "Assert billing reminder email notification event is queued"
      ],
      logs: [
        "CRON JOB: Scanning active trial subscriptions for expiration triggers...",
        "SCHEDULER: Identified subscription 'sub_77182' expiration date reached: 2026-06-29T22:00:00Z",
        "DATABASE: UPDATE subscriptions SET status = 'canceled' WHERE id = 'sub_77182'",
        "API TEST: Requesting POST /api/v1/streams/start (Auth: user_77182)",
        "ASSERT: Asserting API response is HTTP 403 Forbidden - Success",
        "ASSERT: Asserting response message contains 'Subscription expired' - Success",
```

```
      "QUEUE: Dispatched billing_reminder email job to Redis BullMQ",
      "RESULT: Expiration logic validated. 4/4 Assertions Passed."
    ]
  },
  {
    id: "SC-BILL-03",
    name: "PayPal Premium Refund & Cancellation",
    gateway: "PayPal",
    plan: "Premium",
    action: "Refund",
    status: "idle",
    assertions: [
      "Assert paypal webhook 'PAYMENT.SALE.REFUNDED' is signed & verified",
      "Assert invoice status in billing_history transitions to 'refunded'",
      "Assert client account access is restricted to 'free_trial' limits",
      "Assert system triggers tax adjustment calculations ($0.00 tax due)"
    ],
    logs: [
      "PAYPAL MOCK: Dispatching event 'PAYMENT.SALE.REFUNDED' for sale ID 'sale_883219'",
      "WEBHOOK: Verifying PayPal webhook signature headers - Valid",
      "DATABASE: UPDATE billing_history SET status = 'refunded' WHERE invoice_id = 'inv_39821'",
      "DATABASE: UPDATE subscriptions SET plan = 'free_trial', channel_limit = 1 WHERE user_id = 'usr_10283'",
      "ASSERT: Checking active stream count - Active streams: 0 - Clean",
      "ASSERT: Tax write-back calculation complete. Tax adjusted: -$2.90 - Passed",
      "RESULT: Refund lifecycle test verified. 4/4 Assertions Passed."
    ]
  }
]);

const [activeLogs, setActiveLogs] = useState<string[]>([]);
const [activeScenarioName, setActiveScenarioName] = useState<string>("");
const [runningId, setRunningId] = useState<string | null>(null);

const runScenario = async (index: number) => {
  const sc = scenarios[index];
  setRunningId(sc.id);
  setScenarios(prev => {
    const next = [...prev];
    next[index].status = "running";
    return next;
  });

  setActiveScenarioName(sc.name);
  setActiveLogs([`Initiating Billing Test Scenario: ${sc.name}...`]);

  for (let i = 0; i < sc.logs.length; i++) {
    await new Promise(resolve => setTimeout(resolve, 500));
    setActiveLogs(prev => [...prev, sc.logs[i]]);
  }

  setScenarios(prev => {
    const next = [...prev];
    next[index].status = "passed";
    return next;
  });
  setRunningId(null);
};

const planLimits = {
  "Free Trial": { channels: 1, storage: "5 GB", streamTime: "10 hours", price: "$0" },
  "Standard": { channels: 1, storage: "20 GB", streamTime: "50 hours", price: "$19/mo" },
  "Premium": { channels: 3, storage: "100 GB", streamTime: "250 hours", price: "$49/mo" },
  "Pro": { channels: 6, storage: "500 GB", streamTime: "Unlimited", price: "$99/mo" }
};
```

```
  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div>
        <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
          <CreditCard className="w-8 h-8 text-primary" />
          Subscription & Billing Testing Module
        </h1>
        <p className="text-slate-400 mt-1 font-medium">Verify webhook integrations, payment gateways, plans limits
enforcing, and subscription state machines.</p>
      </div>

      {/* Plans Limits Reference Guide */}
      <div className="glass-panel p-6 rounded-2xl">
        <h2 className="text-base font-bold text-white mb-4 flex items-center gap-2">
          <Layers className="w-4 h-4 text-secondary" />
          SaaS Tiers & System Enforcement Limits
        </h2>
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
          {Object.entries(planLimits).map(([planName, limits]) => (
            <div key={planName} className="p-4 rounded-xl bg-slate-900/30 border border-[var(--border)] relative
overflow-hidden">
              <div className="absolute top-2 right-2 text-xs font-bold text-primary px-1.5 py-0.5 rounded
bg-primary/10">
                {limits.price}
              </div>
              <h3 className="font-bold text-white text-sm">{planName}</h3>
              <div className="mt-3 space-y-1.5 text-xs text-slate-400">
                <p>Channels: <span className="text-white font-semibold">{limits.channels}</span></p>
                <p>Storage: <span className="text-white font-semibold">{limits.storage}</span></p>
                <p>Stream Limit: <span className="text-white font-semibold">{limits.streamTime}</span></p>
              </div>
            </div>
          ))}
        </div>
      </div>

      {/* Main Runner Layout */}
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {/* Left: Interactive Scenario Cards */}
        <div className="lg:col-span-2 space-y-6">
          <div className="flex items-center justify-between border-b border-[var(--border)] pb-4">
            <h2 className="text-lg font-bold text-white flex items-center gap-2">
              <Database className="w-5 h-5 text-warning" />
              Webhook & Lifecycle Scenarios
            </h2>
            <span className="text-xs text-slate-400">Gateways: Stripe, Razorpay, PayPal</span>
          </div>

          {scenarios.map((sc, index) => (
            <div key={sc.id} className="glass-panel p-6 rounded-2xl hover:border-slate-700 transition-colors
space-y-4">
              <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3">
                <div>
                  <div className="flex items-center gap-2">
                    <span className="text-[10px] font-bold text-white px-2 py-0.5 rounded bg-white/5 border
border-slate-700">
                      {sc.gateway}
                    </span>
                    <span className="text-[10px] font-bold text-secondary px-2 py-0.5 rounded bg-secondary/10">
                      {sc.action}
                    </span>
```

```
                  <span className="text-[10px] text-slate-500 font-mono">{sc.id}</span>
                </div>
                <h3 className="font-bold text-white text-base mt-2">{sc.name}</h3>
              </div>

              <div className="flex items-center gap-2.5">
                <span className={`px-2 py-0.5 rounded text-xs font-semibold ${
                  sc.status === "passed"
                    ? "bg-success/10 text-success border border-success/20"
                    : sc.status === "running"
                      ? "bg-primary/10 text-primary border border-primary/20"
                      : "bg-slate-800 text-slate-400 border border-slate-700"
                }`}>
                  {sc.status.toUpperCase()}
                </span>

                <button
                  onClick={() => runScenario(index)}
                  disabled={runningId !== null}
                  className="flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold bg-primary
hover:bg-primary-hover text-white transition-colors disabled:opacity-50"
                >
                  <Play className="w-3 h-3" /> Run Verification
                </button>
              </div>
            </div>

            {/* Assertions grid */}
            <div className="bg-slate-950/20 rounded-xl p-4 border border-[var(--border)/5] space-y-2">
              <p className="text-[11px] font-bold text-slate-400 uppercase tracking-wider">Subscription Rules
Validated</p>
              <div className="grid grid-cols-1 sm:grid-cols-2 gap-2 text-xs">
                {sc.assertions.map((assert, i) => (
                  <div key={i} className="flex items-center gap-2 text-slate-300">
                    <span className={`w-1.5 h-1.5 rounded-full shrink-0 ${sc.status === "passed" ? "bg-success" :
"bg-slate-500"}`} />
                    <span className="line-clamp-1">{assert}</span>
                  </div>
                ))}
              </div>
            </div>
          </div>
        ))}
      </div>

      {/* Right: Console Log output */}
      <div>
        <div className="glass-panel p-6 rounded-2xl flex flex-col h-[480px] sticky top-24">
          <h2 className="text-base font-bold text-white mb-4 flex items-center gap-2">
            <DollarSign className="w-4 h-4 text-success" />
            Gateway Payload Verifier
          </h2>

          <div className="flex-1 overflow-y-auto bg-slate-950/80 rounded-xl p-4 font-mono text-[10px] space-y-2
text-slate-300 border border-slate-800 leading-relaxed scrollbar-thin">
            {activeLogs.length === 0 ? (
              <div className="text-slate-500 text-center py-24">
                <p>Trigger a billing scenario to stream payload validations...</p>
              </div>
            ) : (
              <>
                <div className="text-success font-semibold border-b border-white/5 pb-1 mb-2">
                  Running: {activeScenarioName}
                </div>
```

```
                {activeLogs.map((log, index) => (
                  <div key={index} className={
                    log.startsWith("RESULT:")
                      ? "text-success font-bold mt-2 pt-2 border-t border-white/5"
                      : log.startsWith("STRIPE") || log.startsWith("PAYPAL") || log.startsWith("RAZORPAY")
                        ? "text-primary"
                        : log.startsWith("ASSERT:")
                          ? "text-success font-medium"
                          : "text-slate-300"
                  }>
                    {log}
                  </div>
                ))}
              </>
            )}
          </div>

          {activeLogs.length > 0 && (
            <button
              onClick={() => {
                setActiveLogs([]);
                setActiveScenarioName("");
              }}
              className="mt-4 w-full py-2 bg-white/5 hover:bg-white/10 text-xs font-semibold text-slate-300
hover:text-white rounded-xl border border-[var(--border)] transition-colors"
            >
              Clear Output
            </button>
          )}
        </div>
      </div>

    </div>
  </div>
);
}
```

File: src/app/channels-testing/page.tsx

```
"use client";

import React, { useState } from "react";
import { Tv, ShieldCheck, Check, Play, RefreshCw, AlertCircle, RefreshCcw } from "lucide-react";

interface OAuthChannel {
  id: string;
  platform: "YouTube" | "Facebook";
  channelName: string;
  scopes: string[];
  status: "connected" | "expired" | "scope_missing";
  tokenLifeSeconds: number;
}

export default function ChannelsTesting() {
  const [channels, setChannels] = useState<OAuthChannel[]>([
    {
      id: "ch-yt-102",
      platform: "YouTube",
      channelName: "TechSolutions Broadcasters",
      scopes: ["youtube.force-ssl", "youtube.readonly", "userinfo.profile"],
      status: "connected",
      tokenLifeSeconds: 3590
    },
    {
      id: "ch-fb-204",
      platform: "Facebook",
      channelName: "TechSolutions Media Page",
      scopes: ["pages_manage_posts", "pages_read_engagement", "publish_video"],
      status: "scope_missing",
      scopesMissing: ["pages_show_list"],
      statusDetails: "Access token is valid, but missing 'pages_show_list' scope required to list pages."
    } as any,
    {
      id: "ch-yt-103",
      platform: "YouTube",
      channelName: "Gaming Zone Live Stream",
      scopes: ["youtube.force-ssl"],
      status: "expired",
      statusDetails: "OAuth Access Token expired at 2026-06-29T21:40:00Z."
    }
  ]);

  const [activeLogs, setActiveLogs] = useState<string[]>([]);
  const [activeChannelName, setActiveChannelName] = useState<string>("");
  const [testingId, setTestingId] = useState<string | null>(null);

  const runChannelTest = async (channel: OAuthChannel, index: number) => {
    setTestingId(channel.id);
    setActiveChannelName(`${channel.platform} (${channel.channelName})`);
    setActiveLogs([`Initiating Integration Verification for ${channel.platform} channel...`]);

    const delay = (ms: number) => new Promise(r => setTimeout(r, ms));

    await delay(300);
    setActiveLogs(prev => [...prev, `[OAuth] Checking client signature credentials...`]);
    await delay(300);
    setActiveLogs(prev => [...prev, `[OAuth] Verifying access token status: ${channel.status}...`]);

    if (channel.status === "expired") {
      setActiveLogs(prev => [...prev, `[WARN] Token expired. Triggering OAuth Refresh Token flow...`]);
```

```
      await delay(600);
      setActiveLogs(prev => [...prev, `[OAuth] Refreshing token using endpoint: oauth2.googleapis.com/token...`]);
      await delay(500);
      setActiveLogs(prev => [...prev, `[OAuth] New access token generated successfully. Valid for 3600 seconds.`]);

      // Update status in list
      setChannels(prev => {
        const next = [...prev];
        next[index].status = "connected";
        next[index].tokenLifeSeconds = 3600;
        return next;
      });
    } else if (channel.status === "scope_missing") {
      setActiveLogs(prev => [...prev, `[ERROR] Scope checks failed. Missing scope: pages_show_list`]);
      await delay(400);
      setActiveLogs(prev => [...prev, `[API] Attempting Page Retrieval query without scope...`]);
      await delay(300);
      setActiveLogs(prev => [...prev, `[API] Graph API response: HTTP 403 Forbidden (Code 200: Permissions error)`]);
      setActiveLogs(prev => [...prev, `RESULT: Integration Failed. Scopes must be refreshed by user.`]);
      setTestingId(null);
      return;
    }

    await delay(400);
    setActiveLogs(prev => [...prev, `[API] Generating scheduled live broadcast metadata payload...`]);
    await delay(400);

    if (channel.platform === "YouTube") {
      setActiveLogs(prev => [...prev, `[YouTube API] POST https://www.googleapis.com/youtube/v3/liveBroadcasts`]);
      await delay(300);
      setActiveLogs(prev => [...prev, `[YouTube API] Response: HTTP 201 Created | Broadcast ID: yt_live_901923`]);
      await delay(200);
      setActiveLogs(prev => [...prev, `[YouTube API] POST https://www.googleapis.com/youtube/v3/liveStreams`]);
      await delay(300);
      setActiveLogs(prev => [...prev, `[YouTube API] Response: RTMP Ingest URL: rtmp://a.rtmp.youtube.com/live2 |
Stream Key: ****-****-****`]);
    } else {
      setActiveLogs(prev => [...prev, `[Facebook API] POST https://graph.facebook.com/v18.0/{page_id}/live_videos`]);
      await delay(500);
      setActiveLogs(prev => [...prev, `[Facebook API] Response: HTTP 200 OK | Video ID: fb_live_109283921 | Secure
RTMP URL: rtmps://live-api-s.facebook.com:443/rtmp/...`]);
    }

    await delay(200);
    setActiveLogs(prev => [...prev, `RESULT: Social API handshake completed successfully! Integration verified.`]);
    setTestingId(null);
  };

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div>
        <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
          <Tv className="w-8 h-8 text-primary" />
          Channel Integration Testing
        </h1>
        <p className="text-slate-400 mt-1 font-medium">Validate OAuth token refresh loops, scopes alignment, live
broadcast creations, and stream settings endpoints.</p>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {/* Left Column: Channels */}
        <div className="lg:col-span-2 space-y-6">
```

```
          <div className="flex items-center justify-between border-b border-[var(--border)] pb-4">
            <h2 className="text-lg font-bold text-white">OAuth Integrations Registry</h2>
            <span className="text-xs text-slate-400">Sandbox Environment</span>
          </div>

          {channels.map((channel, idx) => (
            <div key={channel.id} className="glass-panel p-6 rounded-2xl space-y-4">
              <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3">
                <div>
                  <div className="flex items-center gap-2.5">
                    <span className={`text-[10px] font-bold text-white px-2 py-0.5 rounded ${
                      channel.platform === "YouTube" ? "bg-red-600" : "bg-blue-600"
                    }`}>
                      {channel.platform}
                    </span>
                    <span className="text-[10px] text-slate-500 font-mono">{channel.id}</span>
                  </div>
                  <h3 className="font-bold text-white text-base mt-2">{channel.channelName}</h3>
                </div>

                <div className="flex items-center gap-2">
                  <span className={`px-2.5 py-0.5 rounded text-xs font-semibold ${
                    channel.status === "connected"
                      ? "bg-success/10 text-success border border-success/20"
                      : channel.status === "expired"
                        ? "bg-warning/10 text-warning border border-warning/20"
                        : "bg-danger/10 text-danger border border-danger/20"
                  }`}>
                    {channel.status === "connected" ? "CONNECTED" : channel.status === "expired" ? "EXPIRED" : "SCOPE
ERROR"}
                  </span>

                  <button
                    onClick={() => runChannelTest(channel, idx)}
                    disabled={testingId !== null}
                    className="flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold bg-primary
hover:bg-primary-hover text-white transition-colors disabled:opacity-50"
                  >
                    {channel.status === "expired" ? (
                      <>
                        <RefreshCcw className="w-3 h-3" /> Refresh & Test
                      </>
                    ) : (
                      <>
                        <Play className="w-3 h-3" /> Verify Handshake
                      </>
                    )}
                  </button>
                </div>
              </div>

              {/* Scopes Section */}
              <div className="bg-slate-950/20 rounded-xl p-4 border border-[var(--border)/5] space-y-2 text-xs">
                <p className="text-[10px] font-bold text-slate-400 uppercase tracking-wider">Authorized Permission
Scopes</p>
                <div className="flex flex-wrap gap-1.5 pt-1">
                  {channel.scopes.map(scope => (
                    <span key={scope} className="px-2.5 py-0.5 rounded bg-white/5 border border-[var(--border)]
text-slate-300 font-mono text-[10px]">
                      {scope}
                    </span>
                  ))}
                  {channel.status === "scope_missing" && (channel as any).scopesMissing?.map((scope: string) => (
                    <span key={scope} className="px-2.5 py-0.5 rounded bg-danger/10 border border-danger/20
```

```
text-danger font-mono text-[10px]">
                      {scope} (Missing)
                    </span>
                  ))}
                </div>
                {channel.statusDetails && (
                  <p className="text-slate-400 text-xs mt-2 italic flex items-center gap-1.5 text-[11px]">
                    <AlertCircle className="w-3.5 h-3.5 text-warning shrink-0" />
                    {channel.statusDetails}
                  </p>
                )}
              </div>
            </div>
          ))}

          {/* Future Channels Grid */}
          <div className="glass-panel p-6 rounded-2xl">
            <h3 className="text-sm font-bold text-slate-300 mb-3">Integrations Under Development (Mock Environments
Available)</h3>
            <div className="grid grid-cols-2 sm:grid-cols-4 gap-3 text-xs">
              {["Instagram Live", "Twitch API", "TikTok Live", "LinkedIn Live"].map(plat => (
                <div key={plat} className="p-3 rounded-xl bg-white/2 border border-[var(--border)] text-slate-400
text-center font-medium">
                  {plat}
                </div>
              ))}
            </div>
          </div>
        </div>

        {/* Right Column: API handshake logs */}
        <div>
          <div className="glass-panel p-6 rounded-2xl flex flex-col h-[500px] sticky top-24">
            <h2 className="text-base font-bold text-white mb-4 flex items-center gap-2">
              <ShieldCheck className="w-4 h-4 text-success" />
              API Handshake Verifier
            </h2>

            <div className="flex-1 overflow-y-auto bg-slate-950/80 rounded-xl p-4 font-mono text-[10px] space-y-2
text-slate-300 border border-slate-800 leading-relaxed scrollbar-thin">
              {activeLogs.length === 0 ? (
                <div className="text-slate-500 text-center py-28">
                  <p>Trigger a handshake to verify API request payloads...</p>
                </div>
              ) : (
                <>
                  <div className="text-success font-semibold border-b border-white/5 pb-1 mb-2">
                    Running: {activeChannelName}
                  </div>
                  {activeLogs.map((log, index) => (
                    <div key={index} className={
                      log.startsWith("RESULT:")
                        ? "text-success font-bold mt-2 pt-2 border-t border-white/5"
                        : log.startsWith("[OAuth]")
                          ? "text-primary"
                          : log.startsWith("[ERROR]") || log.includes("Forbidden")
                            ? "text-danger"
                            : log.startsWith("[WARN]")
                              ? "text-warning"
                              : "text-slate-300"
                    }>
                      {log}
                    </div>
                  ))}
```

```
                </>
              )}
            </div>

            {activeLogs.length > 0 && (
              <button
                onClick={() => {
                  setActiveLogs([]);
                  setActiveChannelName("");
                }}
                className="mt-4 w-full py-2 bg-white/5 hover:bg-white/10 text-xs font-semibold text-slate-300
hover:text-white rounded-xl border border-[var(--border)] transition-colors"
              >
                Clear Handshake Console
              </button>
            )}
          </div>
        </div>

      </div>
    </div>
  );
}
```

File: src/app/streaming-testing/page.tsx

```
"use client";

import React, { useEffect, useState } from "react";
import { Activity, Play, AlertOctagon, RefreshCw, Terminal, CheckCircle2, ShieldAlert, Cpu } from "lucide-react";

export default function StreamingTesting() {
  const [engineState, setEngineState] = useState<"streaming" | "failed" | "retrying" | "recovered">("streaming");
  const [retryCount, setRetryCount] = useState(0);
  const [backoffDelay, setBackoffDelay] = useState(0);
  const [logs, setLogs] = useState<string[]>([
    "[16:54:10] [Engine] Spawning stream ingest worker...",
    "[16:54:11] [Worker-01] Stream PID 14092 created. Outputting RTMP to destinations...",
    "[16:54:12] [Monitor] Frame drop check: 0 frames dropped. Network Jitter: 4ms.",
    "[16:55:00] [Monitor] Frame drop check: 2 frames dropped. Bitrate: 4850 kbps."
  ]);
  const [isProcessing, setIsProcessing] = useState(false);

  const appendLog = (msg: string) => {
    setLogs(prev => [...prev, `[${new Date().toLocaleTimeString()}] ${msg}`]);
  };

  const triggerNetworkInterrupt = async () => {
    setIsProcessing(true);
    setEngineState("failed");
    appendLog("[CRITICAL] RTMP Network drop detected! Ingest packets stalled.");

    // Step 1: Wait and go to retry
    await new Promise(r => setTimeout(r, 1200));
    setEngineState("retrying");
    setRetryCount(1);
    setBackoffDelay(2); // 2^1 seconds
    appendLog("[RETRY-ENGINE] Attempt 1: Calculating exponential backoff (2^1 = 2 seconds delay)...");

    await new Promise(r => setTimeout(r, 2000));
    appendLog("[RETRY-ENGINE] Dispatching connection probe to YouTube ingest server...");
    appendLog("[WARN] Ingest server unreachable. Network route timeout.");

    // Step 2: Retry 2
    setRetryCount(2);
    setBackoffDelay(4); // 2^2 seconds
    appendLog("[RETRY-ENGINE] Attempt 2: Calculating backoff (2^2 = 4 seconds delay)...");

    await new Promise(r => setTimeout(r, 3000)); // wait slightly shorter for UI speed
    appendLog("[RETRY-ENGINE] Probing network route. Latency: 180ms. Link active!");
    appendLog("[RETRY-ENGINE] Re-authorizing RTMP credentials...");

    await new Promise(r => setTimeout(r, 1000));
    setEngineState("recovered");
    setRetryCount(0);
    setBackoffDelay(0);
    appendLog("[RECOVERY] Stream source pipe recovered. Resuming FFmpeg frame push.");
    appendLog("[Monitor] Frame rate stabilized at 30fps. Bitrate: 4800 kbps.");
    setIsProcessing(false);
  };

  const triggerWorkerCrash = async () => {
    setIsProcessing(true);
    setEngineState("failed");
    appendLog("[CRITICAL] Container FFmpeg-worker-01 killed (SIGKILL). Process PID 14092 terminated.");

    // BullMQ and Redis orchestration simulation
```

```
    await new Promise(r => setTimeout(r, 1000));
    setEngineState("retrying");
    appendLog("[BullMQ] Job 'stream_process_usr_90123' set to retrying state. Redelivering queue ticket.");

    await new Promise(r => setTimeout(r, 1200));
    appendLog("[Orchestrator] Provisioning replacement pod: worker-node-02-pod.");
    appendLog("[Orchestrator] Pod status: RUNNING. Spawning FFmpeg PID 18902.");

    await new Promise(r => setTimeout(r, 800));
    setEngineState("recovered");
    appendLog("[RECOVERY] Worker node handoff completed. Ingest stream recovered successfully.");
    setIsProcessing(false);
  };

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div>
        <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
          <Activity className="w-8 h-8 text-primary" />
          Live Streaming Engine Testing
        </h1>
        <p className="text-slate-400 mt-1 font-medium">Inject synthetic connection faults, kill encoding containers,
and verify exponential backoff retry state machine.</p>
      </div>

      {/* State Graph and Stats */}
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {/* Left: State Engine Visualizer */}
        <div className="lg:col-span-2 space-y-6">
          <div className="glass-panel p-6 rounded-2xl">
            <h2 className="text-base font-bold text-white mb-6 flex items-center gap-2">
              <Cpu className="w-4 h-4 text-secondary" />
              State Machine Visualizer & Failure Injector
            </h2>

            {/* Simulated state flow chart */}
            <div className="flex flex-col sm:flex-row items-center justify-center gap-4 py-8 bg-slate-950/20
rounded-2xl border border-[var(--border)/5] mb-8">
              <div className={`px-4 py-2.5 rounded-xl border text-xs font-bold transition-all ${
                engineState === "streaming"
                  ? "bg-success/15 text-success border-success glow-success"
                  : "bg-white/5 border-[var(--border)] text-slate-500"
              }`}>
                1. STREAMING (OK)
              </div>
              <div className="hidden sm:block text-slate-600">-></div>

              <div className={`px-4 py-2.5 rounded-xl border text-xs font-bold transition-all ${
                engineState === "failed"
                  ? "bg-danger/15 text-danger border-danger glow-primary"
                  : "bg-white/5 border-[var(--border)] text-slate-500"
              }`}>
                2. FAILED (FAULT)
              </div>
              <div className="hidden sm:block text-slate-600">-></div>

              <div className={`px-4 py-2.5 rounded-xl border text-xs font-bold transition-all ${
                engineState === "retrying"
                  ? "bg-warning/15 text-warning border-warning glow-primary animate-pulse"
                  : "bg-white/5 border-[var(--border)] text-slate-500"
              }`}>
                3. RETRYING ({retryCount > 0 ? `Attempt ${retryCount}` : "WAIT"})
```

```
              </div>
              <div className="hidden sm:block text-slate-600">-></div>

              <div className={`px-4 py-2.5 rounded-xl border text-xs font-bold transition-all ${
                engineState === "recovered"
                  ? "bg-primary/15 text-primary border-primary glow-secondary"
                  : "bg-white/5 border-[var(--border)] text-slate-500"
              }`}>
                4. RECOVERED
              </div>
            </div>

            {/* Backoff Info */}
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 mb-6">
              <div className="p-4 rounded-xl bg-slate-900/30 border border-[var(--border)]">
                <p className="text-xs text-slate-400 font-medium">Exponential Backoff Formula</p>
                <p className="text-lg font-bold text-white font-mono mt-1">Delay = min(2<sup>retry</sup>, 64)s</p>
                <p className="text-[10px] text-slate-500 mt-2">Protects ingestion nodes from request floods during
recovery.</p>
              </div>
              <div className="p-4 rounded-xl bg-slate-900/30 border border-[var(--border)]">
                <p className="text-xs text-slate-400 font-medium">Active Backoff Countdown</p>
                <p className="text-xl font-bold text-warning font-mono mt-1">
                  {backoffDelay > 0 ? `${backoffDelay} seconds remaining` : "0 seconds (Idle)"}
                </p>
                <p className="text-[10px] text-slate-500 mt-2">Simulated timer representing client wait window.</p>
              </div>
            </div>

            {/* Control buttons */}
            <div className="space-y-3">
              <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Fault Injection Controllers</p>
              <div className="flex flex-wrap gap-3">
                <button
                  onClick={triggerNetworkInterrupt}
                  disabled={isProcessing}
                  className="flex items-center gap-2 px-5 py-2.5 rounded-xl bg-danger hover:bg-red-600 text-white
font-bold text-xs shadow-lg glow-primary disabled:opacity-50 transition-colors"
                >
                  <AlertOctagon className="w-4 h-4" /> Inject 15s Network Outage
                </button>
                <button
                  onClick={triggerWorkerCrash}
                  disabled={isProcessing}
                  className="flex items-center gap-2 px-5 py-2.5 rounded-xl bg-gradient-to-r from-orange-500
to-amber-500 hover:opacity-90 text-white font-bold text-xs shadow-lg disabled:opacity-50 transition-colors"
                >
                  <AlertOctagon className="w-4 h-4" /> Inject FFmpeg Worker Crash
                </button>
                <button
                  onClick={() => {
                    setEngineState("streaming");
                    setRetryCount(0);
                    setBackoffDelay(0);
                    setLogs([
                      "[16:54:10] [Engine] Spawning stream Ingest worker...",
                      "[16:54:11] [Worker-01] Stream PID 14092 created. Outputting RTMP to destinations...",
                      "[16:54:12] [Monitor] Frame drop check: 0 frames dropped. Network Jitter: 4ms."
                    ]);
                  }}
                  disabled={isProcessing}
                  className="flex items-center gap-2 px-5 py-2.5 rounded-xl bg-white/5 border border-[var(--border)]
hover:bg-white/10 text-slate-300 hover:text-white font-bold text-xs disabled:opacity-50 transition-colors"
                >
```

```
                <RefreshCw className="w-4 h-4" /> Reset Engine Simulator
              </button>
            </div>
          </div>

        </div>
      </div>

      {/* Right: Ingest Console log output */}
      <div>
        <div className="glass-panel p-6 rounded-2xl flex flex-col h-[500px] sticky top-24">
          <div className="flex items-center justify-between border-b border-[var(--border)] pb-4 mb-4">
            <h2 className="text-base font-bold text-white flex items-center gap-2">
              <Terminal className="w-4 h-4 text-primary" />
              FFmpeg Worker Ingest
            </h2>
            <span className="text-[10px] font-mono text-success">ONLINE</span>
          </div>

          <div className="flex-1 overflow-y-auto bg-slate-950/80 rounded-xl p-4 font-mono text-[10px] space-y-2
text-slate-300 border border-slate-800 leading-relaxed scrollbar-thin">
            {logs.map((log, index) => (
              <div key={index} className={
                log.includes("[CRITICAL]")
                  ? "text-danger font-semibold"
                  : log.includes("[RECOVERY]")
                    ? "text-success font-bold mt-1"
                    : log.includes("[RETRY-ENGINE]")
                      ? "text-warning"
                      : "text-slate-300"
              }>
                {log}
              </div>
            ))}
          </div>

          <button
            onClick={() => setLogs([])}
            className="mt-4 w-full py-2 bg-white/5 hover:bg-white/10 text-xs font-semibold text-slate-300
hover:text-white rounded-xl border border-[var(--border)] transition-colors"
          >
            Clear Log Console
          </button>
        </div>
      </div>

    </div>
  </div>
);
}
```

File: src/app/ai-generator/page.tsx

```
"use client";

import React, { useState } from "react";
import { Bot, Play, Sparkles, Code, FileCode, CheckCircle, HelpCircle, RefreshCw } from "lucide-react";

interface TestCase {
  id: string;
  title: string;
  severity: string;
  steps: string[];
  assertions: string[];
  edgeCase: string;
}

interface TestSuite {
  suiteName: string;
  testType: string;
  generatedAt: string;
  testCases: TestCase[];
}

export default function AITestCaseGenerator() {
  const [prompt, setPrompt] = useState("");
  const [testType, setTestType] = useState("functional");
  const [isGenerating, setIsGenerating] = useState(false);
  const [suite, setSuite] = useState<TestSuite | null>(null);

  const templates = [
    {
      title: "YouTube OAuth Integration",
      prompt: "Validate YouTube OAuth connection flow. Ensure refresh token is fetched and cached, and verifying that expired access tokens trigger automated token renewal without interrupting active RTMP broadcasts."
    },
    {
      title: "Stripe Webhook Upgrades",
      prompt: "Validate standard to premium subscription upgrades. Trigger stripe customer.subscription.updated webhook and assert channel limits raise from 1 to 3 in memory, and storage expands to 100GB."
    },
    {
      title: "Worker Failures & Auto-Recovery",
      prompt: "Inject worker failure during live encoding. Assert that primary worker crash (SIGKILL) trigger BullMQ retry scheduler to spin up secondary ffmpeg worker within 2.5 seconds, matching segments."
    }
  ];

  const handleGenerate = async (selectedPrompt?: string) => {
    const activePrompt = selectedPrompt || prompt;
    if (!activePrompt.trim()) return;

    setIsGenerating(true);
    setSuite(null);

    try {
      const res = await fetch("/api/ai-generator", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ prompt: activePrompt, type: testType })
      });
      const data = await res.json();
      if (data.success) {
        setSuite(data);
```

```
      }
    } catch (e) {
      console.error("Error generating test suite:", e);
    } finally {
      setIsGenerating(false);
    }
  };

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div>
        <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
          <Bot className="w-8 h-8 text-primary" />
          AI Test Case Generator
        </h1>
        <p className="text-slate-400 mt-1 font-medium">Draft test cases, integration scenarios, regression specs, and
critical edge cases using NLP prompt analysis.</p>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {/* Left Column: Input Prompt Specs (1/3 width) */}
        <div className="space-y-6">
          <div className="glass-panel p-6 rounded-2xl space-y-4">
            <h2 className="text-base font-bold text-white flex items-center gap-2">
              <Sparkles className="w-4 h-4 text-primary" />
              Prompt Specifications
            </h2>

            <div className="space-y-2">
              <label className="text-xs font-semibold text-slate-400">Describe feature or API endpoint</label>
              <textarea
                value={prompt}
                onChange={(e) => setPrompt(e.target.value)}
                placeholder="e.g. Test password reset flow with expired verification tokens..."
                className="w-full h-32 rounded-xl bg-slate-950/40 border border-[var(--border)] p-3 text-xs text-white
focus:outline-none focus:border-primary placeholder-slate-500"
              />
            </div>

            <div className="space-y-2">
              <label className="text-xs font-semibold text-slate-400">Test Class Type</label>
              <select
                value={testType}
                onChange={(e) => setTestType(e.target.value)}
                className="w-full rounded-xl bg-slate-900 border border-[var(--border)] p-3 text-xs text-white
focus:outline-none focus:border-primary"
              >
                <option value="functional">Functional Validation</option>
                <option value="integration">API & Integration</option>
                <option value="security">Security Vulnerability Check</option>
                <option value="performance">Load & Scale Test</option>
                <option value="regression">Regression Suite</option>
              </select>
            </div>

            <button
              onClick={() => handleGenerate()}
              disabled={isGenerating || !prompt.trim()}
              className="w-full flex items-center justify-center gap-2 py-3 rounded-xl bg-gradient-to-r from-primary
to-secondary hover:opacity-90 text-white font-bold text-xs shadow-lg glow-primary disabled:opacity-50
transition-colors"
            >
```

```
              {isGenerating ? (
                <>
                  <RefreshCw className="w-4 h-4 animate-spin" />
                  Analyzing Specs...
                </>
              ) : (
                <>
                  <Bot className="w-4 h-4" />
                  Generate Test Suite
                </>
              )}
            </button>
          </div>

          {/* Quick templates */}
          <div className="glass-panel p-6 rounded-2xl space-y-4">
            <h3 className="text-xs font-bold text-slate-400 uppercase tracking-wider">Example System Templates</h3>
            <div className="space-y-2.5">
              {templates.map(tpl => (
                <button
                  key={tpl.title}
                  onClick={() => {
                    setPrompt(tpl.prompt);
                    handleGenerate(tpl.prompt);
                  }}
                  className="w-full text-left p-3 rounded-xl bg-white/2 hover:bg-white/5 border border-[var(--border)]
text-xs transition-colors group"
                >
                  <p className="font-bold text-white group-hover:text-primary transition-colors">{tpl.title}</p>
                  <p className="text-[11px] text-slate-400 mt-1 line-clamp-2 leading-relaxed">{tpl.prompt}</p>
                </button>
              ))}
            </div>
          </div>
        </div>

        {/* Right Column: AI Output (2/3 width) */}
        <div className="lg:col-span-2 space-y-6">
          {!suite && !isGenerating && (
            <div className="glass-panel p-12 rounded-2xl flex flex-col items-center justify-center text-center
space-y-4 h-full min-h-[400px]">
              <div className="w-16 h-16 rounded-full bg-slate-900 border border-[var(--border)] flex items-center
justify-center text-slate-400">
                <Bot className="w-8 h-8" />
              </div>
              <div>
                <h3 className="font-bold text-white text-base">Awaiting Specifications</h3>
                <p className="text-xs text-slate-400 mt-1 max-w-sm">Enter specifications or select a quick template to
trigger AI-powered test case generation.</p>
              </div>
            </div>
          )}

          {isGenerating && (
            <div className="glass-panel p-12 rounded-2xl flex flex-col items-center justify-center text-center
space-y-4 h-full min-h-[400px]">
              <div className="w-12 h-12 rounded-full border-2 border-primary border-t-transparent animate-spin flex
items-center justify-center" />
              <div>
                <h3 className="font-bold text-white text-base">Generative Engine Running</h3>
                <p className="text-xs text-slate-400 mt-1 max-w-xs">Parsing user story dependencies, mapping database
rules, and formatting code assertions...</p>
              </div>
            </div>
```

```
          )}

          {suite && (
            <div className="space-y-6">
              {/* Meta banner */}
              <div className="glass-panel p-5 rounded-2xl flex flex-col sm:flex-row sm:items-center justify-between
gap-3 bg-gradient-to-r from-primary/10 to-transparent border-primary/20">
                <div>
                  <div className="flex items-center gap-2">
                    <span className="text-[9px] uppercase font-bold tracking-wider px-2 py-0.5 rounded bg-primary/20
border border-primary/30 text-white">
                      {suite.testType}
                    </span>
                    <span className="text-[10px] text-slate-500">Generated: {new
Date(suite.generatedAt).toLocaleTimeString()}</span>
                  </div>
                  <h2 className="text-lg font-bold text-white mt-1.5">{suite.suiteName}</h2>
                </div>
                <div className="flex items-center gap-2">
                  <button className="flex items-center gap-1 text-[11px] font-semibold px-3 py-1.5 bg-white/5 border
border-[var(--border)] text-slate-300 hover:text-white rounded-lg transition-colors">
                    <Code className="w-3.5 h-3.5" /> Export Jest Specs
                  </button>
                  <button className="flex items-center gap-1 text-[11px] font-semibold px-3 py-1.5 bg-white/5 border
border-[var(--border)] text-slate-300 hover:text-white rounded-lg transition-colors">
                    <FileCode className="w-3.5 h-3.5" /> Playwright TS
                  </button>
                </div>
              </div>

              {/* Test Cases Grid */}
              <div className="space-y-6">
                {suite.testCases.map((tc, idx) => (
                  <div key={tc.id} className="glass-panel p-6 rounded-2xl space-y-4">
                    <div className="flex items-center justify-between">
                      <h3 className="font-bold text-white text-base flex items-center gap-2">
                        <span className="text-slate-500 text-xs font-mono">{tc.id}</span>
                        {tc.title}
                      </h3>
                      <span className="text-[10px] font-bold text-danger px-2.5 py-0.5 rounded bg-danger/10 border
border-danger/20 uppercase">
                        {tc.severity} Severity
                      </span>
                    </div>

                    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
                      {/* Steps */}
                      <div className="space-y-2">
                        <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Test Steps</p>
                        <ol className="list-decimal list-inside space-y-1.5 text-xs text-slate-300">
                          {tc.steps.map((step, i) => (
                            <li key={i} className="leading-relaxed pl-1">{step}</li>
                          ))}
                        </ol>
                      </div>

                      {/* Assertions */}
                      <div className="space-y-2">
                        <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Assertions</p>
                        <ul className="space-y-1.5 text-xs text-slate-300">
                          {tc.assertions.map((assert, i) => (
                            <li key={i} className="flex items-start gap-2 leading-relaxed">
                              <CheckCircle className="w-4 h-4 text-success shrink-0 mt-0.5" />
                              <span>{assert}</span>
```

```
                          </li>
                        ))}
                      </ul>
                    </div>
                  </div>

                  {/* Edge Case Callout */}
                  <div className="p-4 rounded-xl bg-primary/5 border border-primary/20 flex gap-3 text-xs
leading-normal">
                    <HelpCircle className="w-5 h-5 text-primary shrink-0 mt-0.5" />
                    <div>
                      <p className="font-bold text-primary">AI-Identified Critical Edge Case</p>
                      <p className="text-slate-300 mt-1">{tc.edgeCase}</p>
                    </div>
                  </div>
                </div>
              ))}
            </div>
          </div>
        )}
      </div>

    </div>
  );
}
```

File: src/app/security-scanner/page.tsx

```
"use client";

import React, { useEffect, useState } from "react";
import { ShieldCheck, ShieldAlert, Play, RefreshCw, Terminal, Search, HelpCircle, CheckCircle, AlertTriangle } from
"lucide-react";

interface Finding {
  id: string;
  category: string;
  owasp: string;
  severity: "high" | "medium" | "low";
  endpoint: string;
  description: string;
  evidence: string;
  remediation: string;
}

export default function SecurityScanner() {
  const [endpoint, setEndpoint] = useState("https://api.livestream-platform.example.com");
  const [isScanning, setIsScanning] = useState(false);
  const [progress, setProgress] = useState(0);
  const [riskScore, setRiskScore] = useState(3.4);
  const [findings, setFindings] = useState<Finding[]>([]);
  const [logs, setLogs] = useState<string[]>([]);
  const [activeTab, setActiveTab] = useState<"findings" | "logs">("findings");

  const fetchScanState = async () => {
    try {
      const res = await fetch("/api/security-scan");
      const data = await res.json();
      setRiskScore(data.riskScore);
      setFindings(data.findings);
      if (!isScanning) {
        setLogs(data.logs);
      }
    } catch (e) {
      console.error(e);
    }
  };

  useEffect(() => {
    fetchScanState();
  }, []);

  const runSecurityScan = async () => {
    setIsScanning(true);
    setProgress(0);
    setActiveTab("logs");

    try {
      // Start scan
      await fetch("/api/security-scan", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ action: "start_scan", endpoint })
      });

      // Fetch immediate logs
      const startRes = await fetch("/api/security-scan");
      const startData = await startRes.json();
      setLogs(startData.logs);
```

```
      // Simulate progress ticks
      let currentProgress = 0;
      const interval = setInterval(async () => {
        currentProgress += 20;

        await fetch("/api/security-scan", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ action: "update_progress", progressValue: currentProgress })
        });

        const stateRes = await fetch("/api/security-scan");
        const stateData = await stateRes.json();

        setProgress(currentProgress);
        setLogs(stateData.logs);

        if (currentProgress >= 100) {
          clearInterval(interval);
          setIsScanning(false);
          setRiskScore(stateData.riskScore);
          setFindings(stateData.findings);
          setActiveTab("findings");
        }
      }, 1000);

    } catch (e) {
      console.error(e);
      setIsScanning(false);
    }
  };

  const handleReset = async () => {
    try {
      await fetch("/api/security-scan", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ action: "reset" })
      });
      fetchScanState();
      setProgress(0);
    } catch (e) {
      console.error(e);
    }
  };

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
            <ShieldCheck className="w-8 h-8 text-primary" />
            Security Testing Agent
          </h1>
          <p className="text-slate-400 mt-1 font-medium">Scan API endpoints for OWASP Top 10 vulnerabilities, verify
JWT token lifetimes, and audit file upload MIME restrictions.</p>
        </div>
        <div className="flex items-center gap-2">
          <button
            onClick={handleReset}
            className="px-4 py-2 rounded-xl text-xs font-semibold bg-white/5 border border-[var(--border)]
hover:bg-white/10 text-slate-300 transition-colors"
```

```
          >
            Reset Scanner
          </button>
        </div>
      </div>

      {/* Configuration Panel */}
      <div className="glass-panel p-6 rounded-2xl">
        <h2 className="text-base font-bold text-white mb-4">Target API Configuration</h2>
        <div className="flex flex-col md:flex-row items-center gap-4">
          <div className="relative flex-1 w-full">
            <Search className="absolute left-3.5 top-3.5 w-4.5 h-4.5 text-slate-400" />
            <input
              type="text"
              value={endpoint}
              onChange={(e) => setEndpoint(e.target.value)}
              disabled={isScanning}
              className="w-full rounded-xl bg-slate-950/40 border border-[var(--border)] pl-10 pr-4 py-3 text-xs
text-white focus:outline-none focus:border-primary font-mono"
            />
          </div>

          <button
            onClick={runSecurityScan}
            disabled={isScanning || !endpoint}
            className="w-full md:w-auto flex items-center justify-center gap-2 px-6 py-3 rounded-xl bg-danger
hover:bg-red-600 text-white font-bold text-xs shadow-lg glow-primary disabled:opacity-50 transition-colors shrink-0"
          >
            {isScanning ? (
              <>
                <RefreshCw className="w-4 h-4 animate-spin" />
                Auditing ({progress}%)
              </>
            ) : (
              <>
                <Play className="w-4 h-4" />
                Run Vulnerability Audit
              </>
            )}
          </button>
        </div>

        {isScanning && (
          <div className="w-full bg-slate-900 h-1.5 rounded-full overflow-hidden mt-6">
            <div
              className="bg-danger h-full transition-all duration-300"
              style={{ width: `${progress}%` }}
            />
          </div>
        )}
      </div>

      {/* Vulnerabilities & Stats Grid */}
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {/* Risk Score Widget (1/3 width) */}
        <div className="space-y-6">
          <div className="glass-panel p-6 rounded-2xl flex flex-col items-center justify-center text-center">
            <h2 className="text-sm font-bold text-slate-400 uppercase tracking-wider mb-6">Threat Risk Index</h2>

            {/* Circle progress representation */}
            <div className="relative w-36 h-36 flex items-center justify-center mb-6">
              <svg className="w-full h-full transform -rotate-90">
                <circle
```

```
                cx="72"
                cy="72"
                r="62"
                stroke="rgba(255,255,255,0.03)"
                strokeWidth="10"
                fill="transparent"
              />
              <circle
                cx="72"
                cy="72"
                r="62"
                stroke={riskScore > 6 ? "#ef4444" : riskScore > 3 ? "#f59e0b" : "#22c55e"}
                strokeWidth="10"
                fill="transparent"
                strokeDasharray={`${2 * Math.PI * 62}`}
                strokeDashoffset={`${2 * Math.PI * 62 * (1 - riskScore / 10)}`}
                className="transition-all duration-1000 ease-out"
              />
            </svg>
            <div className="absolute text-center">
              <p className="text-3xl font-extrabold text-white">{riskScore.toFixed(1)}</p>
              <p className="text-[10px] text-slate-400 font-semibold uppercase mt-0.5">Out of 10</p>
            </div>
          </div>

          <div className={`px-4 py-1.5 rounded-full text-xs font-bold ${
            riskScore > 6
              ? "bg-danger/10 text-danger border border-danger/20"
              : riskScore > 3
                ? "bg-warning/10 text-warning border border-warning/20"
                : "bg-success/10 text-success border border-success/20"
          }`}>
            {riskScore > 6 ? "CRITICAL THREAT RATING" : riskScore > 3 ? "MEDIUM RISK" : "SECURE"}
          </div>

          <div className="w-full grid grid-cols-3 gap-2 mt-8 text-center text-xs border-t border-[var(--border)]
pt-6">
            <div>
              <p className="text-slate-400">High</p>
              <p className="font-bold text-danger mt-1">
                {findings.filter(f => f.severity === "high").length}
              </p>
            </div>
            <div>
              <p className="text-slate-400">Medium</p>
              <p className="font-bold text-warning mt-1">
                {findings.filter(f => f.severity === "medium").length}
              </p>
            </div>
            <div>
              <p className="text-slate-400">Low</p>
              <p className="font-bold text-success-hover mt-1">
                {findings.filter(f => f.severity === "low").length}
              </p>
            </div>
          </div>
        </div>

        {/* Quick Info card */}
        <div className="glass-panel p-6 rounded-2xl space-y-3">
          <h3 className="text-xs font-bold text-slate-400 uppercase tracking-wider">MIME Upload Scanner</h3>
          <p className="text-xs text-slate-400 leading-relaxed">
            Automated testing scripts intercept mock file uploads and scan for executable patterns (e.g. `.exe`,
`.elf`, `.bat`) disguised as stream MP4 media assets.
```

```
            </p>
            <div className="flex items-center gap-2 text-xs text-success bg-success/5 border border-success/15 p-3
rounded-xl">
              <CheckCircle className="w-4.5 h-4.5 shrink-0" />
              Upload validation filter: ACTIVE
            </div>
          </div>
        </div>

        {/* Audit Details & Logs (2/3 width) */}
        <div className="lg:col-span-2 space-y-6">
          <div className="flex border-b border-[var(--border)]">
            <button
              onClick={() => setActiveTab("findings")}
              className={`px-6 py-3 text-xs font-bold transition-all relative ${
                activeTab === "findings" ? "text-white" : "text-slate-400 hover:text-slate-200"
              }`}
            >
              Audits Findings ({findings.length})
              {activeTab === "findings" && (
                <span className="absolute bottom-0 left-0 right-0 h-[2px] bg-primary" />
              )}
            </button>
            <button
              onClick={() => setActiveTab("logs")}
              className={`px-6 py-3 text-xs font-bold transition-all relative ${
                activeTab === "logs" ? "text-white" : "text-slate-400 hover:text-slate-200"
              }`}
            >
              Scanner Logs
              {activeTab === "logs" && (
                <span className="absolute bottom-0 left-0 right-0 h-[2px] bg-primary" />
              )}
            </button>
          </div>

          {activeTab === "findings" ? (
            <div className="space-y-6">
              {findings.map(finding => (
                <div key={finding.id} className="glass-panel p-6 rounded-2xl space-y-4 hover:border-slate-700
transition-colors">
                  <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3">
                    <div>
                      <div className="flex items-center gap-2">
                        <span className="text-[9px] uppercase font-bold tracking-wider px-2 py-0.5 rounded bg-white/5
border border-slate-700 text-slate-300">
                          {finding.owasp}
                        </span>
                        <span className="text-[10px] text-slate-500 font-mono">{finding.id}</span>
                      </div>
                      <h3 className="font-bold text-white text-base mt-1.5">{finding.category}</h3>
                    </div>

                    <span className={`px-2.5 py-0.5 rounded text-[10px] font-extrabold uppercase w-fit ${
                      finding.severity === "high"
                        ? "bg-danger/10 text-danger border border-danger/20"
                        : finding.severity === "medium"
                          ? "bg-warning/10 text-warning border border-warning/20"
                          : "bg-success/10 text-success border border-success/20"
                    }`}>
                      {finding.severity} Severity
                    </span>
                  </div>
```

```
                <div className="space-y-3.5 text-xs">
                  <div>
                    <p className="text-slate-400 font-semibold">Vulnerable API Endpoint:</p>
                    <p className="font-mono text-white mt-1 bg-slate-950/40 p-2 rounded border
border-[var(--border)] text-[10px] overflow-x-auto select-all">
                      {finding.endpoint}
                    </p>
                  </div>

                  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
                    <div>
                      <p className="text-slate-400 font-semibold">Description:</p>
                      <p className="text-slate-300 mt-1 leading-relaxed">{finding.description}</p>
                    </div>
                    <div>
                      <p className="text-slate-400 font-semibold">Evidence Captured:</p>
                      <p className="font-mono text-amber-400 mt-1 leading-relaxed bg-slate-950/20 p-2 rounded border
border-[var(--border)/5] text-[10px]">
                        {finding.evidence}
                      </p>
                    </div>
                  </div>

                  <div className="p-3.5 rounded-xl bg-primary/5 border border-primary/20 flex gap-2.5
leading-relaxed">
                    <AlertTriangle className="w-4.5 h-4.5 text-primary shrink-0 mt-0.5" />
                    <div>
                      <p className="font-bold text-primary">Remediation Guide</p>
                      <p className="text-slate-300 mt-0.5">{finding.remediation}</p>
                    </div>
                  </div>
                </div>
              </div>
            ))}
          </div>
        ) : (
          <div className="glass-panel p-6 rounded-2xl flex flex-col h-[400px]">
            <div className="flex-1 overflow-y-auto bg-slate-950/80 rounded-xl p-4 font-mono text-[10px] space-y-2
text-slate-300 border border-slate-800 leading-relaxed scrollbar-thin">
              {logs.map((log, index) => (
                <div key={index} className={
                  log.includes("Match found!") || log.includes("vulnerabilities identified")
                    ? "text-danger font-semibold"
                    : log.includes("Scan complete")
                      ? "text-success font-bold mt-2 border-t border-white/5 pt-2"
                      : "text-slate-300"
                }>
                  {log}
                </div>
              ))}
            </div>
          </div>
        )}
      </div>

    </div>
  </div>
);
}
```

File: src/app/load-tester/page.tsx

```
"use client";

import React, { useEffect, useState } from "react";
import { Zap, Play, RefreshCw, BarChart2, ShieldAlert, Cpu, CheckCircle } from "lucide-react";
import {
  LineChart,
  Line,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  Legend,
  ResponsiveContainer,
  BarChart,
  Bar
} from "recharts";

interface MetricRow {
  timestamp: string;
  responseTime: number;
  throughput: number;
  errorRate: number;
  cpu: number;
  memory: number;
}

export default function LoadTester() {
  const [activeTier, setActiveTier] = useState<"1k" | "10k" | "50k" | "100k">("1k");
  const [isRunning, setIsRunning] = useState(false);
  const [progress, setProgress] = useState(0);
  const [chartData, setChartData] = useState<MetricRow[]>([]);
  const [allMetrics, setAllMetrics] = useState<Record<string, MetricRow[]>>({});
  const [summary, setSummary] = useState({
    tool: "k6",
    engine: "Kube-k6-Operator",
    runnerNodes: 8,
    status: "ready"
  });

  const loadData = async () => {
    try {
      const res = await fetch("/api/load-test");
      const data = await res.json();
      setAllMetrics(data.metrics);
      setChartData(data.metrics[activeTier]);
      setSummary(data.summary);
    } catch (e) {
      console.error(e);
    }
  };

  useEffect(() => {
    loadData();
  }, [activeTier]);

  const triggerLoadTest = () => {
    setIsRunning(true);
    setProgress(0);

    // Simulate real-time progress increments
    const interval = setInterval(() => {
```

```
      setProgress(prev => {
        if (prev >= 100) {
          clearInterval(interval);
          setIsRunning(false);
          return 100;
        }
        return prev + 25;
      });
    }, 800);
  };

  // Compute summary stats based on current tier data
  const getStats = () => {
    if (!chartData || chartData.length === 0) return { avgLatency: 0, peakThroughput: 0, totalErrors: "0%" };

    const totalLatency = chartData.reduce((acc, row) => acc + row.responseTime, 0);
    const avgLatency = Math.round(totalLatency / chartData.length);

    const peakThroughput = Math.max(...chartData.map(row => row.throughput));

    const avgError = (chartData.reduce((acc, row) => acc + row.errorRate, 0) / chartData.length).toFixed(2);

    return { avgLatency, peakThroughput, totalErrors: `${avgError}%` };
  };

  const stats = getStats();

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
            <Zap className="w-8 h-8 text-primary" />
            Performance & Load Testing
          </h1>
          <p className="text-slate-400 mt-1 font-medium">Verify API response speeds, requests-per-second ceilings, and
error percentages under simulated user crowds.</p>
        </div>
        <div className="flex items-center gap-2.5">
          <span className="text-xs text-slate-400 bg-white/5 border border-[var(--border)] px-3.5 py-1.5 rounded-xl
font-mono">
            Runner: {summary.tool.toUpperCase()} (Distributed)
          </span>
        </div>
      </div>

      {/* Control panel & KPI cards */}
      <div className="grid grid-cols-1 lg:grid-cols-4 gap-6">

        {/* Run controls */}
        <div className="glass-panel p-6 rounded-2xl flex flex-col justify-between space-y-4">
          <div>
            <h2 className="text-base font-bold text-white">Load Configuration</h2>
            <p className="text-xs text-slate-400 mt-1">Select concurrent users target</p>
          </div>

          <div className="grid grid-cols-2 gap-2">
            {(["1k", "10k", "50k", "100k"] as const).map(tier => (
              <button
                key={tier}
                onClick={() => setActiveTier(tier)}
                disabled={isRunning}
                className={`py-2 rounded-xl text-xs font-bold transition-all ${
```

```
                activeTier === tier
                  ? "bg-primary border border-primary text-white"
                  : "bg-slate-900/40 border border-[var(--border)] text-slate-400 hover:text-white"
              }`}
            >
              {tier.toUpperCase()} Users
            </button>
          ))}
        </div>

        <button
          onClick={triggerLoadTest}
          disabled={isRunning}
          className="w-full flex items-center justify-center gap-2 py-3 rounded-xl bg-gradient-to-r from-primary
to-secondary hover:opacity-90 text-white font-bold text-xs shadow-lg glow-primary disabled:opacity-50 transition-all"
        >
          {isRunning ? (
            <>
              <RefreshCw className="w-4 h-4 animate-spin" />
              Injecting load ({progress}%)
            </>
          ) : (
            <>
              <Play className="w-4 h-4" /> Start Load Test
            </>
          )}
        </button>
      </div>

      {/* Stats card: Latency */}
      <div className="glass-panel p-6 rounded-2xl flex flex-col justify-between">
        <p className="text-xs font-medium text-slate-400 uppercase tracking-wider">Average Latency</p>
        <div className="my-2">
          <p className="text-3xl font-bold text-white">{stats.avgLatency} <span className="text-sm font-normal
text-slate-400">ms</span></p>
        </div>
        <div className="flex items-center gap-1 text-xs text-success font-medium">
          <span className="w-1.5 h-1.5 rounded-full bg-success" />
          Within 500ms API SLA threshold
        </div>
      </div>

      {/* Stats card: Throughput */}
      <div className="glass-panel p-6 rounded-2xl flex flex-col justify-between">
        <p className="text-xs font-medium text-slate-400 uppercase tracking-wider">Peak Throughput</p>
        <div className="my-2">
          <p className="text-3xl font-bold text-white">{stats.peakThroughput.toLocaleString()} <span
className="text-sm font-normal text-slate-400">req/s</span></p>
        </div>
        <div className="flex items-center gap-1 text-xs text-slate-400">
          Distributed across {summary.runnerNodes} pod nodes
        </div>
      </div>

      {/* Stats card: Errors */}
      <div className="glass-panel p-6 rounded-2xl flex flex-col justify-between">
        <p className="text-xs font-medium text-slate-400 uppercase tracking-wider">Average Error Rate</p>
        <div className="my-2">
          <p className={`text-3xl font-bold ${parseFloat(stats.totalErrors) > 2 ? "text-danger" : "text-white"}`}>
            {stats.totalErrors}
          </p>
        </div>
        <div className="flex items-center gap-1.5 text-xs font-medium text-slate-400">
          {parseFloat(stats.totalErrors) > 1.5 ? (
```

```
            <span className="text-danger flex items-center gap-1">
              <ShieldAlert className="w-3.5 h-3.5" /> High threshold warnings
            </span>
          ) : (
            <span className="text-success flex items-center gap-1">
              <CheckCircle className="w-3.5 h-3.5" /> Normal bounds ({"<"} 2.0%)
            </span>
          )}
        </div>
      </div>
    </div>

    {/* Charts Grid */}
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

      {/* Latency & Throughput Timeline Chart (2/3 width) */}
      <div className="lg:col-span-2 glass-panel p-6 rounded-2xl">
        <h2 className="text-lg font-bold text-white mb-6 flex items-center gap-2">
          <BarChart2 className="w-5 h-5 text-primary" />
          Performance latency & requests Timeline
        </h2>

        <div className="h-80 w-full">
          <ResponsiveContainer width="100%" height="100%">
            <LineChart
              data={chartData}
              margin={{ top: 10, right: 10, left: -10, bottom: 0 }}
            >
              <CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.05)" />
              <XAxis dataKey="timestamp" stroke="#94a3b8" fontSize={11} />
              <YAxis yAxisId="left" stroke="#94a3b8" fontSize={11} label={{ value: 'Latency (ms)', angle: -90,
position: 'insideLeft', fill: '#94a3b8', fontSize: 10 }} />
              <YAxis yAxisId="right" orientation="right" stroke="#94a3b8" fontSize={11} label={{ value: 'Throughput
(req/s)', angle: 90, position: 'insideRight', fill: '#94a3b8', fontSize: 10 }} />
              <Tooltip
                contentStyle={{
                  backgroundColor: "#1e293b",
                  borderColor: "rgba(255,255,255,0.08)",
                  borderRadius: "12px",
                  color: "#f8fafc"
                }}
              />
              <Legend wrapperStyle={{ fontSize: 11, paddingTop: 10 }} />
              <Line
                yAxisId="left"
                type="monotone"
                dataKey="responseTime"
                name="Response Time (ms)"
                stroke="#6366f1"
                strokeWidth={2}
                activeDot={{ r: 6 }}
              />
              <Line
                yAxisId="right"
                type="monotone"
                dataKey="throughput"
                name="Throughput (req/s)"
                stroke="#8b5cf6"
                strokeWidth={2}
                activeDot={{ r: 6 }}
              />
            </LineChart>
          </ResponsiveContainer>
        </div>
```

```
        </div>

        {/* System Load Distribution Chart (1/3 width) */}
        <div className="glass-panel p-6 rounded-2xl flex flex-col justify-between">
          <div>
            <h2 className="text-lg font-bold text-white mb-2 flex items-center gap-2">
              <Cpu className="w-5 h-5 text-secondary" />
              Node Stress Index
            </h2>
            <p className="text-xs text-slate-400 mb-6">Resource footprints on API runner pods during peak load</p>
          </div>

          <div className="h-64 w-full">
            <ResponsiveContainer width="100%" height="100%">
              <BarChart
                data={chartData ? [chartData[chartData.length - 1]] : []}
                margin={{ top: 10, right: 10, left: -25, bottom: 0 }}
              >
                <CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.05)" />
                <XAxis dataKey="timestamp" stroke="#94a3b8" fontSize={11} hide />
                <YAxis stroke="#94a3b8" fontSize={11} domain={[0, 100]} />
                <Tooltip
                  contentStyle={{
                    backgroundColor: "#1e293b",
                    borderColor: "rgba(255,255,255,0.08)",
                    borderRadius: "12px",
                    color: "#f8fafc"
                  }}
                />
                <Legend wrapperStyle={{ fontSize: 11 }} />
                <Bar dataKey="cpu" name="CPU Core Peak %" fill="#6366f1" radius={[8, 8, 0, 0]} />
                <Bar dataKey="memory" name="RAM Peak %" fill="#8b5cf6" radius={[8, 8, 0, 0]} />
              </BarChart>
            </ResponsiveContainer>
          </div>
        </div>

      </div>
    </div>
  );
}
```

File: src/app/admin-panel/page.tsx

```
"use client";

import React, { useState } from "react";
import { Settings, ShieldAlert, Users, CreditCard, Tag, TrendingUp, UserMinus, Plus, ShieldCheck } from
"lucide-react";

interface AdminUser {
  id: string;
  name: string;
  email: string;
  plan: string;
  status: "active" | "banned" | "trial_expired";
}

interface Coupon {
  code: string;
  discountType: "percentage" | "fixed";
  value: number;
  status: "active" | "expired";
}

export default function AdminPanel() {
  const [users, setUsers] = useState<AdminUser[]>([
    { id: "usr-01", name: "David Miller", email: "david.m@streamcorp.io", plan: "Pro Plan", status: "active" },
    { id: "usr-02", name: "Sarah Jenkins", email: "s.jenkins@broadcasthub.com", plan: "Standard Plan", status:
"active" },
    { id: "usr-03", name: "Alex Rover", email: "alex.rover@indiecast.tv", plan: "Free Trial", status: "trial_expired"
},
    { id: "usr-04", name: "Bad Actor", email: "spammer_99@throwaway.com", plan: "Free Trial", status: "banned" }
  ]);

  const [coupons, setCoupons] = useState<Coupon[]>([
    { code: "STREAM50", discountType: "percentage", value: 50, status: "active" },
    { code: "SAVE20USD", discountType: "fixed", value: 20, status: "active" }
  ]);

  const [newCouponCode, setNewCouponCode] = useState("");
  const [newCouponType, setNewCouponType] = useState<"percentage" | "fixed">("percentage");
  const [newCouponVal, setNewCouponVal] = useState(15);

  const [alerts, setAlerts] = useState<string[]>([]);

  const triggerAlert = (msg: string) => {
    setAlerts(prev => [msg, ...prev].slice(0, 3));
  };

  const handleBanUser = (id: string, name: string) => {
    setUsers(prev => prev.map(u => {
      if (u.id === id) {
        const isBannedNow = u.status === "banned";
        const nextStatus = isBannedNow ? "active" : "banned";
        triggerAlert(`User '${name}' status updated to: ${nextStatus.toUpperCase()}`);
        return { ...u, status: nextStatus as any };
      }
      return u;
    }));
  };

  const handleExtendTrial = (name: string) => {
    triggerAlert(`Extended trial for '${name}' by 14 additional days. Status updated in subscription DB.`);
  };
```

```
  const handleCreateCoupon = (e: React.FormEvent) => {
    e.preventDefault();
    if (!newCouponCode.trim()) return;

    const newC: Coupon = {
      code: newCouponCode.toUpperCase().replace(/\s+/g, ""),
      discountType: newCouponType,
      value: newCouponVal,
      status: "active"
    };

    setCoupons(prev => [newC, ...prev]);
    triggerAlert(`Created coupon code: ${newC.code} (${newC.value}${newC.discountType === "percentage" ? "%" : " USD"} off)`);
    setNewCouponCode("");
  };

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div>
        <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
          <Settings className="w-8 h-8 text-primary" />
          Admin Portal Testing
        </h1>
        <p className="text-slate-400 mt-1 font-medium">Validate user access overrides, extend trials, create discount campaigns, and audit system logs.</p>
      </div>

      {/* Revenue metrics and Audit notifications */}
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {/* Left Column: Stats & Operations (2/3 width) */}
        <div className="lg:col-span-2 space-y-8">

          {/* Revenue Analytics Widget */}
          <div className="glass-panel p-6 rounded-2xl">
            <h2 className="text-base font-bold text-white mb-4 flex items-center gap-2">
              <TrendingUp className="w-4.5 h-4.5 text-secondary" />
              SaaS MRR Revenue Metrics
            </h2>
            <div className="grid grid-cols-3 gap-4 text-center">
              <div className="p-4 rounded-xl bg-slate-900/30 border border-[var(--border)]">
                <p className="text-xs text-slate-400 font-medium">Monthly Recur. (MRR)</p>
                <p className="text-2xl font-bold text-white mt-1.5">$48,250</p>
              </div>
              <div className="p-4 rounded-xl bg-slate-900/30 border border-[var(--border)]">
                <p className="text-xs text-slate-400 font-medium">User Churn Rate</p>
                <p className="text-2xl font-bold text-white mt-1.5">1.8%</p>
              </div>
              <div className="p-4 rounded-xl bg-slate-900/30 border border-[var(--border)]">
                <p className="text-xs text-slate-400 font-medium">Avg Revenue (ARPU)</p>
                <p className="text-2xl font-bold text-white mt-1.5">$34.50</p>
              </div>
            </div>
          </div>

          {/* User management table */}
          <div className="glass-panel p-6 rounded-2xl">
            <h2 className="text-base font-bold text-white mb-4 flex items-center gap-2">
              <Users className="w-4.5 h-4.5 text-primary" />
              User Profiles Management
            </h2>
```

```
            <div className="overflow-x-auto">
              <table className="w-full text-left text-xs border-collapse">
                <thead>
                  <tr className="border-b border-[var(--border)] text-slate-400 font-medium">
                    <th className="pb-3.5 pl-2">User / Email</th>
                    <th className="pb-3.5">Active Plan</th>
                    <th className="pb-3.5">Status</th>
                    <th className="pb-3.5 text-right pr-2">Actions</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-[var(--border)/5]">
                  {users.map(user => (
                    <tr key={user.id} className="hover:bg-white/2 transition-colors">
                      <td className="py-3.5 pl-2">
                        <p className="font-bold text-white">{user.name}</p>
                        <p className="text-[10px] text-slate-500 font-mono mt-0.5">{user.email}</p>
                      </td>
                      <td className="py-3.5 text-slate-300 font-medium">{user.plan}</td>
                      <td className="py-3.5">
                        <span className={`px-2 py-0.5 rounded text-[10px] font-bold ${
                          user.status === "active"
                            ? "bg-success/10 text-success border border-success/20"
                            : user.status === "banned"
                              ? "bg-danger/10 text-danger border border-danger/20"
                              : "bg-slate-800 text-slate-400 border border-slate-700"
                        }`}>
                          {user.status.toUpperCase()}
                        </span>
                      </td>
                      <td className="py-3.5 text-right pr-2 space-x-2">
                        {user.status === "trial_expired" && (
                          <button
                            onClick={() => handleExtendTrial(user.name)}
                            className="px-2.5 py-1 rounded bg-secondary/15 hover:bg-secondary/25 text-secondary border
border-secondary/20 transition-all font-bold text-[10px]"
                          >
                            Extend Trial
                          </button>
                        )}
                        <button
                          onClick={() => handleBanUser(user.id, user.name)}
                          className={`px-2.5 py-1 rounded transition-all font-bold text-[10px] ${
                            user.status === "banned"
                              ? "bg-success/15 hover:bg-success/25 text-success border border-success/20"
                              : "bg-danger/15 hover:bg-danger/25 text-danger border border-danger/20"
                          }`}
                        >
                          {user.status === "banned" ? "Unban" : "Ban User"}
                        </button>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          </div>
        </div>

        {/* Right Column: Coupon Creator & Live DB Hooks log (1/3 width) */}
        <div className="space-y-6">
          {/* Coupon Creator */}
          <div className="glass-panel p-6 rounded-2xl">
            <h2 className="text-base font-bold text-white mb-4 flex items-center gap-2">
```

```
              <Tag className="w-4.5 h-4.5 text-warning" />
              Campaign Coupons Generator
            </h2>

            <form onSubmit={handleCreateCoupon} className="space-y-4">
              <div className="space-y-1.5">
                <label className="text-[11px] font-semibold text-slate-400">Coupon Promo Code</label>
                <input
                  type="text"
                  placeholder="e.g. SUMMERSAVE30"
                  value={newCouponCode}
                  onChange={(e) => setNewCouponCode(e.target.value)}
                  className="w-full rounded-xl bg-slate-950/40 border border-[var(--border)] p-3 text-xs text-white
focus:outline-none focus:border-primary font-mono uppercase"
                />
              </div>

              <div className="grid grid-cols-2 gap-3">
                <div className="space-y-1.5">
                  <label className="text-[11px] font-semibold text-slate-400">Type</label>
                  <select
                    value={newCouponType}
                    onChange={(e) => setNewCouponType(e.target.value as any)}
                    className="w-full rounded-xl bg-slate-900 border border-[var(--border)] p-3 text-xs text-white
focus:outline-none focus:border-primary"
                  >
                    <option value="percentage">Percent (%)</option>
                    <option value="fixed">Fixed (USD)</option>
                  </select>
                </div>
                <div className="space-y-1.5">
                  <label className="text-[11px] font-semibold text-slate-400">Discount Value</label>
                  <input
                    type="number"
                    value={newCouponVal}
                    onChange={(e) => setNewCouponVal(parseInt(e.target.value) || 0)}
                    className="w-full rounded-xl bg-slate-950/40 border border-[var(--border)] p-3 text-xs text-white
focus:outline-none focus:border-primary font-mono"
                  />
                </div>
              </div>

              <button
                type="submit"
                disabled={!newCouponCode.trim()}
                className="w-full flex items-center justify-center gap-2 py-3 rounded-xl bg-gradient-to-r from-primary
to-secondary hover:opacity-90 text-white font-bold text-xs shadow-lg glow-primary disabled:opacity-50
transition-colors"
              >
                <Plus className="w-4 h-4" /> Create Coupon
              </button>
            </form>

            {/* Coupons List */}
            <div className="mt-6 border-t border-[var(--border)] pt-4 space-y-2">
              <p className="text-[10px] font-bold text-slate-400 uppercase tracking-wider mb-2">Active Coupons
Campaign</p>
              {coupons.map(c => (
                <div key={c.code} className="flex items-center justify-between text-xs p-2.5 rounded-lg
bg-slate-950/20 border border-[var(--border)/5]">
                  <span className="font-mono font-bold text-white">{c.code}</span>
                  <span className="text-slate-400">
                    {c.value}{c.discountType === "percentage" ? "%" : " USD"} off
                  </span>
```

```
                </div>
              ))}
            </div>
          </div>

          {/* Administrative Audit Logs Hook */}
          <div className="glass-panel p-6 rounded-2xl flex flex-col h-[280px]">
            <h2 className="text-base font-bold text-white mb-4 flex items-center gap-2">
              <ShieldAlert className="w-4.5 h-4.5 text-danger" />
              Database Hooks Audit
            </h2>

            <div className="flex-1 overflow-y-auto bg-slate-950/80 rounded-xl p-4 font-mono text-[9px] space-y-2
text-slate-300 border border-slate-800 leading-relaxed scrollbar-thin">
              {alerts.length === 0 ? (
                <div className="text-slate-500 text-center py-16">
                  <p>Awaiting administrative actions...</p>
                </div>
              ) : (
                alerts.map((msg, idx) => (
                  <div key={idx} className="text-slate-300 border-b border-white/5 pb-1.5 last:border-b-0">
                    <span className="text-primary font-bold">[{new Date().toLocaleTimeString()}] DB_HOOK:</span> {msg}
                  </div>
                ))
              )}
            </div>
          </div>
        </div>

      </div>
    </div>
  );
}
```

File: src/app/reports/page.tsx

```
"use client";

import React, { useState } from "react";
import { FileSpreadsheet, Download, FileText, CheckCircle2, ShieldCheck, Zap, KeyRound } from "lucide-react";

interface ReportItem {
  id: string;
  name: string;
  type: "Security Audit" | "Performance Benchmark" | "E2E Auth Suite" | "Channel Handshake";
  date: string;
  status: "Passed" | "Warnings" | "Failed";
  downloadFileName: string;
  csvData: string;
}

export default function Reports() {
  const [reports] = useState<ReportItem[]>([
    {
      id: "REP-SEC-01",
      name: "Q2-2026 API OWASP Security Vulnerability Audit",
      type: "Security Audit",
      date: "2026-06-29 16:57:42",
      status: "Warnings",
      downloadFileName: "streamtest_security_audit_q2_2026.csv",
      csvData: "Vulnerability ID,Category,OWASP Code,Severity,Status,Remediation\n" +
               "vuln-01,Broken Object Level Authorization (BOLA),API1:2023,High,Open,Implement tenancy check\n" +
               "vuln-02,Cross-Site Scripting (Stored XSS),A03:2021-Injection,Medium,Open,Sanitize descriptions
input\n" +
               "vuln-03,Rate Limit Missing,A04:2021-Design,Low,Open,Implement IP rate limiter\n"
    },
    {
      id: "REP-PERF-02",
      name: "100,000 Users Distributed k6 Load Test Report",
      type: "Performance Benchmark",
      date: "2026-06-29 16:55:12",
      status: "Passed",
      downloadFileName: "streamtest_k6_load_test_100k.csv",
      csvData: "Timestamp,Concurrent Users,Avg Response Time (ms),Throughput (req/s),Error Rate (%)\n" +
               "0s,100000,340,78000,0.65\n" +
               "10s,100000,395,89000,1.12\n" +
               "20s,100000,445,95500,1.84\n" +
               "30s,100000,520,98200,2.45\n" +
               "40s,100000,498,99100,2.10\n" +
               "50s,100000,565,97500,3.12\n" +
               "60s,100000,590,98800,3.56\n"
    },
    {
      id: "REP-AUTH-03",
      name: "Authentication & Lockout Policy E2E Log",
      type: "E2E Auth Suite",
      date: "2026-06-29 16:52:10",
      status: "Passed",
      downloadFileName: "streamtest_auth_e2e_results.csv",
      csvData: "Test Case ID,Assertion Checked,Result,Execution Time (ms)\n" +
               "TC-AUTH-S1,Email Validation format error,Passed,420\n" +
               "TC-AUTH-S1,Existing email duplicate constraint,Passed,310\n" +
               "TC-AUTH-L1,Lockout after 5 invalid attempts,Passed,890\n" +
               "TC-AUTH-G1,OAuth callback validation token,Passed,670\n"
    }
  ]);
```

```
  const handleDownloadCSV = (report: ReportItem) => {
    const blob = new Blob([report.csvData], { type: "text/csv;charset=utf-8;" });
    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.setAttribute("href", url);
    link.setAttribute("download", report.downloadFileName);
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  };

  const getStatusStyle = (status: string) => {
    if (status === "Passed") return "bg-success/10 text-success border border-success/20";
    if (status === "Warnings") return "bg-warning/10 text-warning border border-warning/20";
    return "bg-danger/10 text-danger border border-danger/20";
  };

  const getTypeIcon = (type: string) => {
    if (type === "Security Audit") return <ShieldCheck className="w-5 h-5 text-danger" />;
    if (type === "Performance Benchmark") return <Zap className="w-5 h-5 text-primary" />;
    return <KeyRound className="w-5 h-5 text-secondary" />;
  };

  return (
    <div className="space-y-8 animate-fade-in-up">
      {/* Header */}
      <div>
        <h1 className="text-3xl font-extrabold text-white tracking-tight flex items-center gap-3">
          <FileSpreadsheet className="w-8 h-8 text-primary" />
          System Reports Portal
        </h1>
        <p className="text-slate-400 mt-1 font-medium">Export performance benchmarks, security audit metrics, and E2E
test results directly to your local computer.</p>
      </div>

      {/* Reports Directory */}
      <div className="glass-panel p-6 rounded-2xl">
        <div className="flex items-center justify-between border-b border-[var(--border)] pb-4 mb-6">
          <h2 className="text-lg font-bold text-white flex items-center gap-2">
            <FileSpreadsheet className="w-5 h-5 text-secondary" />
            QA Reports Directory
          </h2>
          <span className="text-xs text-slate-400">Select format to export</span>
        </div>

        <div className="space-y-4">
          {reports.map((report) => (
            <div
              key={report.id}
              className="p-5 rounded-2xl bg-slate-900/40 border border-[var(--border)] hover:border-slate-700
transition-colors flex flex-col sm:flex-row sm:items-center justify-between gap-4"
            >
              <div className="flex items-start gap-4">
                <div className="p-3 rounded-xl bg-slate-950/60 border border-[var(--border)/5] shrink-0">
                  {getTypeIcon(report.type)}
                </div>
                <div>
                  <div className="flex items-center gap-2">
                    <span className="text-[10px] font-mono text-slate-500">{report.id}</span>
                    <span className="text-[10px] text-slate-400 font-semibold">* {report.date}</span>
                  </div>
                  <h3 className="font-bold text-white text-sm mt-1.5 leading-snug">{report.name}</h3>
                  <p className="text-[11px] text-slate-400 mt-0.5">Type: {report.type}</p>
                </div>
```

```
              </div>

              <div className="flex items-center gap-3 shrink-0">
                <span className={`px-2.5 py-0.5 rounded text-xs font-semibold ${getStatusStyle(report.status)}`}>
                  {report.status.toUpperCase()}
                </span>

                <button
                  onClick={() => handleDownloadCSV(report)}
                  className="flex items-center gap-1.5 px-4 py-2.5 rounded-xl text-xs font-bold bg-primary
hover:bg-primary-hover text-white transition-all shadow-lg glow-primary"
                >
                  <Download className="w-3.5 h-3.5" /> Export CSV
                </button>

                <button
                  onClick={() => alert(`Generating PDF layout for ${report.id}. Standard print stylesheet will
open.`)}
                  className="flex items-center gap-1.5 px-4 py-2.5 rounded-xl text-xs font-bold bg-white/5 border
border-[var(--border)] hover:bg-white/10 text-slate-300 transition-colors"
                >
                  <FileText className="w-3.5 h-3.5" /> PDF
                </button>
              </div>
            </div>
          ))}
        </div>
      </div>

      {/* Integration details */}
      <div className="glass-panel p-6 rounded-2xl border-dashed bg-gradient-to-tr from-primary/5 to-transparent">
        <h3 className="text-sm font-bold text-slate-300 mb-2">Continuous Testing Automated Exports</h3>
        <p className="text-xs text-slate-400 leading-relaxed max-w-2xl">
          Integrate webhooks in your CI/CD pipeline to push these reports directly to Slack, Microsoft Teams, or email
digests upon every release cycle. Check out our <span className="text-primary hover:underline cursor-pointer">CI/CD
documentation</span> for code samples.
        </p>
      </div>
    </div>
  );
}
```

File: src/app/api/health/route.ts

```
import { NextResponse } from "next/server";

export async function GET() {
  return NextResponse.json({
    status: "healthy",
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    services: {
      database: "connected",
      redis: "connected",
      workers: "active (8/8)"
    }
  });
}
```

File: src/app/api/stream-engine/route.ts

```
import { NextResponse } from "next/server";

// Keep a simple mock state in-memory (this is a dev server mock)
let activeStreams = [
  {
    id: "str-90123",
    title: "Global Tech Summit Keynote 2026",
    video: "tech_keynote_final_1080p.mp4",
    channels: ["YouTube", "Facebook"],
    scheduledTime: "Live Now",
    status: "streaming",
    fps: 30,
    bitrate: 4850,
    resolution: "1080p",
    uptime: "02:14:15",
    frameDrops: 12,
    networkStatus: "stable",
    retryCount: 0
  },
  {
    id: "str-90124",
    title: "Product Launch: StreamTest AI Demo",
    video: "streamtest_launch.mp4",
    channels: ["YouTube", "Twitch"],
    scheduledTime: "Live Now",
    status: "streaming",
    fps: 29.97,
    bitrate: 5900,
    resolution: "1080p",
    uptime: "00:45:10",
    frameDrops: 4,
    networkStatus: "stable",
    retryCount: 0
  }
];

let streamEngineLogs = [
  "[16:54:10] [Engine] Initializing RTMP pipelines...",
  "[16:54:11] [Worker-01] Spawning FFmpeg process PID: 14092",
  "[16:54:12] [YouTube-Destination] Connecting to RTMP ingest: a.rtmp.youtube.com/live2...",
  "[16:54:13] [Facebook-Destination] Connecting to RTMP ingest: rtmp-api.facebook.com/live...",
  "[16:54:14] [Engine] HLS segmenter initialized. TS chunks generating successfully.",
  "[16:54:15] [Monitor] Uptime 00:00:01 | Bitrate 4800 kbps | 0 frame drops | Network: EXCELLENT",
  "[16:55:00] [Monitor] Uptime 00:00:46 | Bitrate 4850 kbps | 2 frame drops | Network: EXCELLENT"
];

let workerStatus = {
  cpuUsage: 42.5,
  ramUsage: 58.2,
  storageUsage: 31.8,
  redisQueue: 0,
  activeWorkers: 4,
  ffmpegProcesses: 2
};

export async function GET() {
  return NextResponse.json({
    streams: activeStreams,
    logs: streamEngineLogs,
    system: workerStatus,
    timestamp: new Date().toISOString()
```

```
  });
}

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { action, streamId } = body;

    const streamIndex = activeStreams.findIndex(s => s.id === streamId);

    if (action === "crash_worker") {
      workerStatus.cpuUsage = 98.9;
      workerStatus.activeWorkers = 3;
      workerStatus.ffmpegProcesses = 1;

      if (streamIndex !== -1) {
        activeStreams[streamIndex].status = "failed";
        activeStreams[streamIndex].networkStatus = "disconnected";
        activeStreams[streamIndex].bitrate = 0;
        activeStreams[streamIndex].fps = 0;
      }

      streamEngineLogs.push(`[${new Date().toLocaleTimeString()}] [CRITICAL] FFmpeg Worker Node 02 crashed
unexpectedly! PID 14092 terminated.`);
      streamEngineLogs.push(`[${new Date().toLocaleTimeString()}] [WARN] Stream ${streamId || "str-90123"} has failed.
Triggering Auto Recovery Engine.`);

      return NextResponse.json({ success: true, message: "Simulated worker crash triggered." });
    }

    if (action === "recover_worker") {
      workerStatus.cpuUsage = 40.1;
      workerStatus.activeWorkers = 4;
      workerStatus.ffmpegProcesses = 2;

      if (streamIndex !== -1) {
        activeStreams[streamIndex].status = "streaming";
        activeStreams[streamIndex].networkStatus = "stable";
        activeStreams[streamIndex].bitrate = 4750;
        activeStreams[streamIndex].fps = 30;
        activeStreams[streamIndex].retryCount += 1;
      }

      streamEngineLogs.push(`[${new Date().toLocaleTimeString()}] [RECOVERY] Spawning new replacement worker node. PID
18442.`);
      streamEngineLogs.push(`[${new Date().toLocaleTimeString()}] [RECOVERY] Re-connecting RTMP bindings to YouTube &
Facebook.`);
      streamEngineLogs.push(`[${new Date().toLocaleTimeString()}] [RECOVERY] Stream ${streamId || "str-90123"}
recovered successfully in 4.2 seconds.`);

      return NextResponse.json({ success: true, message: "Simulated recovery executed." });
    }

    if (action === "reset") {
      // Restore initial values
      activeStreams = [
        {
          id: "str-90123",
          title: "Global Tech Summit Keynote 2026",
          video: "tech_keynote_final_1080p.mp4",
          channels: ["YouTube", "Facebook"],
          scheduledTime: "Live Now",
          status: "streaming",
          fps: 30,
```

```
          bitrate: 4850,
          resolution: "1080p",
          uptime: "02:14:15",
          frameDrops: 12,
          networkStatus: "stable",
          retryCount: 0
        },
        {
          id: "str-90124",
          title: "Product Launch: StreamTest AI Demo",
          video: "streamtest_launch.mp4",
          channels: ["YouTube", "Twitch"],
          scheduledTime: "Live Now",
          status: "streaming",
          fps: 29.97,
          bitrate: 5900,
          resolution: "1080p",
          uptime: "00:45:10",
          frameDrops: 4,
          networkStatus: "stable",
          retryCount: 0
        }
      ];
      workerStatus = {
        cpuUsage: 42.5,
        ramUsage: 58.2,
        storageUsage: 31.8,
        redisQueue: 0,
        activeWorkers: 4,
        ffmpegProcesses: 2
      };
      streamEngineLogs = [
        "[16:54:10] [Engine] Initializing RTMP pipelines...",
        "[16:54:11] [Worker-01] Spawning FFmpeg process PID: 14092",
        "[16:54:12] [YouTube-Destination] Connecting to RTMP ingest: a.rtmp.youtube.com/live2...",
        "[16:54:13] [Facebook-Destination] Connecting to RTMP ingest: rtmp-api.facebook.com/live...",
        "[16:54:14] [Engine] HLS segmenter initialized. TS chunks generating successfully."
      ];
      return NextResponse.json({ success: true, message: "Engine simulator reset complete." });
    }

    return NextResponse.json({ error: "Unknown action" }, { status: 400 });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File: src/app/api/security-scan/route.ts

```
import { NextResponse } from "next/server";

let scanState = {
  isScanning: false,
  progress: 0,
  targetEndpoint: "https://api.livestream-platform.example.com",
  riskScore: 3.4, // out of 10
  findings: [
    {
      id: "vuln-01",
      category: "Broken Object Level Authorization (BOLA)",
      owasp: "API1:2023",
      severity: "high",
      endpoint: "/api/v1/channels/{channelId}/billing",
      description: "Endpoint allows standard users to read billing structures of other channels by modifying the
channelId parameter in the request URI.",
      evidence: "HTTP 200 returned for unauthorized channel ID query",
      remediation: "Implement strict tenancy checking on resource ownership in the backend access handler."
    },
    {
      id: "vuln-02",
      category: "Cross-Site Scripting (Stored XSS)",
      owasp: "A03:2021-Injection",
      severity: "medium",
      endpoint: "/api/v1/streams/metadata",
      description: "The stream description input does not sanitize HTML tags before database persistence, which
renders in the public HLS player page layout.",
      evidence: "Injected payload <script>alert(1)</script> was stored and rendered",
      remediation: "Sanitize all incoming string input payloads using DOMPurify or equivalent backend library."
    },
    {
      id: "vuln-03",
      category: "Rate Limit Missing (Broken Authenticated API)",
      owasp: "A04:2021-Design",
      severity: "low",
      endpoint: "/api/v1/auth/forgot-password",
      description: "No rate limit is enforced on password reset mail trigger endpoint, allowing mass mail spamming and
potential server fatigue.",
      evidence: "Sent 500 requests in 10 seconds; all returned HTTP 200 Success",
      remediation: "Implement IP and User-based rate limiting on sensitive transactional endpoints."
    }
  ],
  logs: [
    "Scan ready. Awaiting trigger..."
  ]
};

export async function GET() {
  return NextResponse.json(scanState);
}

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { action, endpoint } = body;

    if (action === "start_scan") {
      scanState.isScanning = true;
      scanState.progress = 0;
      scanState.targetEndpoint = endpoint || scanState.targetEndpoint;
      scanState.logs = [
```

```
        `[${new Date().toLocaleTimeString()}] [Core] Initiating security scan on target:
${scanState.targetEndpoint}...`,
        `[${new Date().toLocaleTimeString()}] [OWASP-A01] Checking Broken Access Control bindings...`,
        `[${new Date().toLocaleTimeString()}] [OWASP-A03] Running SQL Injection and XSS fuzzing scripts...`,
        `[${new Date().toLocaleTimeString()}] [API-Security] Verifying JWT signature validations and token lifetime
rules...`,
        `[${new Date().toLocaleTimeString()}] [Upload-Security] Checking file mime-type verification filters...`
      ];

      return NextResponse.json({ success: true, message: "Scan started." });
    }

    if (action === "update_progress") {
      const { progressValue } = body;
      scanState.progress = progressValue;

      if (progressValue >= 100) {
        scanState.isScanning = false;
        scanState.progress = 100;

        // Add some more logs
        scanState.logs.push(`[${new Date().toLocaleTimeString()}] [OWASP-A03] Match found! Stored XSS vulnerable input
detected at /api/v1/streams/metadata.`);
        scanState.logs.push(`[${new Date().toLocaleTimeString()}] [Core] Scan complete. 3 vulnerabilities identified.
Overall risk score: 6.8 / 10.`);

        // Update findings with a new one to simulate scanning
        scanState.riskScore = 6.8;
      } else {
        scanState.logs.push(`[${new Date().toLocaleTimeString()}] [Scanner] Testing payloads... progress:
${progressValue}%`);
      }

      return NextResponse.json({ success: true, progress: scanState.progress });
    }

    if (action === "reset") {
      scanState = {
        isScanning: false,
        progress: 0,
        targetEndpoint: "https://api.livestream-platform.example.com",
        riskScore: 3.4,
        findings: [
          {
            id: "vuln-01",
            category: "Broken Object Level Authorization (BOLA)",
            owasp: "API1:2023",
            severity: "high",
            endpoint: "/api/v1/channels/{channelId}/billing",
            description: "Endpoint allows standard users to read billing structures of other channels by modifying the
channelId parameter in the request URI.",
            evidence: "HTTP 200 returned for unauthorized channel ID query",
            remediation: "Implement strict tenancy checking on resource ownership in the backend access handler."
          },
          {
            id: "vuln-02",
            category: "Cross-Site Scripting (Stored XSS)",
            owasp: "A03:2021-Injection",
            severity: "medium",
            endpoint: "/api/v1/streams/metadata",
            description: "The stream description input does not sanitize HTML tags before database persistence, which
renders in the public HLS player page layout.",
            evidence: "Injected payload <script>alert(1)</script> was stored and rendered",
            remediation: "Sanitize all incoming string input payloads using DOMPurify or equivalent backend library."
```

```
          },
          {
            id: "vuln-03",
            category: "Rate Limit Missing (Broken Authenticated API)",
            owasp: "A04:2021-Design",
            severity: "low",
            endpoint: "/api/v1/auth/forgot-password",
            description: "No rate limit is enforced on password reset mail trigger endpoint, allowing mass mail
spamming and potential server fatigue.",
            evidence: "Sent 500 requests in 10 seconds; all returned HTTP 200 Success",
            remediation: "Implement IP and User-based rate limiting on sensitive transactional endpoints."
          }
        ],
        logs: [
          "Scan ready. Awaiting trigger..."
        ]
      };
      return NextResponse.json({ success: true, message: "Security scanner simulator reset." });
    }

    return NextResponse.json({ error: "Unknown action" }, { status: 400 });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```

File: src/app/api/load-test/route.ts

```
import { NextResponse } from "next/server";

export async function GET() {
  // Generate load testing series data
  const loadTestTiers = {
    "1k": [
      { timestamp: "0s", responseTime: 120, throughput: 950, errorRate: 0.00, cpu: 12, memory: 35 },
      { timestamp: "10s", responseTime: 125, throughput: 980, errorRate: 0.00, cpu: 14, memory: 35 },
      { timestamp: "20s", responseTime: 118, throughput: 1000, errorRate: 0.00, cpu: 15, memory: 36 },
      { timestamp: "30s", responseTime: 122, throughput: 990, errorRate: 0.02, cpu: 14, memory: 36 },
      { timestamp: "40s", responseTime: 120, throughput: 1005, errorRate: 0.00, cpu: 16, memory: 36 },
      { timestamp: "50s", responseTime: 121, throughput: 1010, errorRate: 0.00, cpu: 15, memory: 36 },
      { timestamp: "60s", responseTime: 123, throughput: 1000, errorRate: 0.00, cpu: 15, memory: 36 }
    ],
    "10k": [
      { timestamp: "0s", responseTime: 140, throughput: 8200, errorRate: 0.01, cpu: 32, memory: 48 },
      { timestamp: "10s", responseTime: 148, throughput: 9500, errorRate: 0.02, cpu: 38, memory: 49 },
      { timestamp: "20s", responseTime: 152, throughput: 9850, errorRate: 0.04, cpu: 41, memory: 51 },
      { timestamp: "30s", responseTime: 145, throughput: 9990, errorRate: 0.03, cpu: 42, memory: 52 },
      { timestamp: "40s", responseTime: 151, throughput: 10050, errorRate: 0.05, cpu: 44, memory: 52 },
      { timestamp: "50s", responseTime: 149, throughput: 10120, errorRate: 0.02, cpu: 43, memory: 52 },
      { timestamp: "60s", responseTime: 153, throughput: 10020, errorRate: 0.04, cpu: 42, memory: 52 }
    ],
    "50k": [
      { timestamp: "0s", responseTime: 210, throughput: 38000, errorRate: 0.12, cpu: 65, memory: 72 },
      { timestamp: "10s", responseTime: 235, throughput: 44200, errorRate: 0.18, cpu: 71, memory: 74 },
      { timestamp: "20s", responseTime: 258, throughput: 48500, errorRate: 0.25, cpu: 78, memory: 76 },
      { timestamp: "30s", responseTime: 270, throughput: 49800, errorRate: 0.32, cpu: 81, memory: 78 },
      { timestamp: "40s", responseTime: 262, throughput: 50100, errorRate: 0.28, cpu: 80, memory: 78 },
      { timestamp: "50s", responseTime: 285, throughput: 49500, errorRate: 0.41, cpu: 83, memory: 79 },
      { timestamp: "60s", responseTime: 290, throughput: 50050, errorRate: 0.38, cpu: 82, memory: 79 }
    ],
    "100k": [
      { timestamp: "0s", responseTime: 340, throughput: 78000, errorRate: 0.65, cpu: 88, memory: 86 },
      { timestamp: "10s", responseTime: 395, throughput: 89000, errorRate: 1.12, cpu: 91, memory: 88 },
      { timestamp: "20s", responseTime: 445, throughput: 95500, errorRate: 1.84, cpu: 94, memory: 89 },
      { timestamp: "30s", responseTime: 520, throughput: 98200, errorRate: 2.45, cpu: 96, memory: 91 },
      { timestamp: "40s", responseTime: 498, throughput: 99100, errorRate: 2.10, cpu: 95, memory: 91 },
      { timestamp: "50s", responseTime: 565, throughput: 97500, errorRate: 3.12, cpu: 97, memory: 92 },
      { timestamp: "60s", responseTime: 590, throughput: 98800, errorRate: 3.56, cpu: 98, memory: 93 }
    ]
  };

  return NextResponse.json({
    metrics: loadTestTiers,
    timestamp: new Date().toISOString(),
    summary: {
      tool: "k6",
      engine: "Distributed Kube-k6-Operator",
      vUs: [1000, 10000, 50000, 100000],
      runnerNodes: 8,
      status: "ready"
    }
  });
}
```

File: src/app/api/ai-generator/route.ts

```
import { NextResponse } from "next/server";

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { prompt, type } = body;

    if (!prompt) {
      return NextResponse.json({ error: "Prompt is required" }, { status: 400 });
    }

    // Delay a bit to simulate AI processing
    await new Promise(resolve => setTimeout(resolve, 800));

    // Generate test cases based on prompt keywords
    const lowerPrompt = prompt.toLowerCase();

    let suiteName = "Custom Integration Test Suite";
    let testType = type || "functional";
    let testCases = [];

    if (lowerPrompt.includes("auth") || lowerPrompt.includes("login") || lowerPrompt.includes("signup")) {
      suiteName = "AI-Generated Authentication Validation Suite";
      testType = "security & functional";
      testCases = [
        {
          id: "TC-AUTH-01",
          title: "Brute-Force Lockout Policy Validation",
          severity: "high",
          steps: [
            "Attempt sign-in with user 'qa_test_brute@streamtest.ai' and random passwords 5 consecutive times.",
            "Inspect user account status via administrative API endpoints.",
            "Attempt a 6th login with the CORRECT password."
          ],
          assertions: [
            "Assert that the 5th attempt triggers a rate-limit block or lockout warning.",
            "Assert that the administrative API reports user state as 'locked_out'.",
            "Assert that the 6th login with correct password returns HTTP 423 Locked."
          ],
          edgeCase: "Verify if lockout timer resets accurately after exactly 15 minutes."
        },
        {
          id: "TC-AUTH-02",
          title: "OAuth Token Hijack & Refresh Boundary Check",
          severity: "critical",
          steps: [
            "Connect channel with Google OAuth and capture access token.",
            "Trigger a token expiration event manually by altering expiration timestamp in local DB.",
            "Dispatch a live stream command immediately post-expiration."
          ],
          assertions: [
            "Assert that the streaming engine intercepts expired token and triggers refresh flow.",
            "Assert that the new access token is updated silently in security context.",
            "Assert that live stream creation command completes successfully with HTTP 201."
          ],
          edgeCase: "Trigger token refresh when Google Auth server reports temporary 503 Service Unavailable."
        }
      ];
    } else if (lowerPrompt.includes("subscription") || lowerPrompt.includes("billing") ||
lowerPrompt.includes("stripe")) {
      suiteName = "AI-Generated Subscription Billing Test Suite";
```

```
      testType = "integration & regression";
      testCases = [
        {
          id: "TC-BILL-01",
          title: "Stripe Webhook Concurrency - Upgrade during active stream",
          severity: "high",
          steps: [
            "Spawn active live stream on Standard Plan (limited to 1 channel).",
            "Simulate concurrent Stripe webhook event 'customer.subscription.updated' upgrading user to Premium (3
channels).",
            "Attempt to connect Facebook Live stream immediately while YouTube is broadcasting."
          ],
          assertions: [
            "Assert that subscription tier state transitions to 'premium' in database within 100ms.",
            "Assert that the channel limit restriction is lifted in active memory state.",
            "Assert that Facebook stream starts successfully without dropping the existing YouTube stream."
          ],
          edgeCase: "Stripe webhook arrives out of order (e.g., 'invoice.payment_failed' arrives after
'invoice.payment_succeeded')."
        },
        {
          id: "TC-BILL-02",
          title: "Grace Period Billing Cycle Logic Verification",
          severity: "medium",
          steps: [
            "Mark billing invoice payment state as failed.",
            "Verify subscription switches to 'past_due' status.",
            "Keep active streams running and attempt to schedule a future stream."
          ],
          assertions: [
            "Assert active streams continue running during the 3-day grace period.",
            "Assert that attempts to schedule a stream beyond the grace period date are blocked.",
            "Verify invoice emails are triggered daily during past_due status."
          ],
          edgeCase: "User downgrades plan to 'free' while account is in 'past_due' state."
        }
      ];
    } else if (lowerPrompt.includes("stream") || lowerPrompt.includes("rtmp") || lowerPrompt.includes("ffmpeg")) {
      suiteName = "AI-Generated Streaming Infrastructure Test Suite";
      testType = "infrastructure & reliability";
      testCases = [
        {
          id: "TC-STRM-01",
          title: "Worker Failure Interception and Seamless RTMP Handover",
          severity: "critical",
          steps: [
            "Initialize live stream broadcasting of 2GB MP4 video to YouTube destination.",
            "After 30s of active broadcasting, kill the active worker node process (simulated SIGKILL).",
            "Observe the HLS playback segment sequence from the player client."
          ],
          assertions: [
            "Assert that the orchestrator detects worker drop within 1.5 seconds.",
            "Assert that a secondary worker pulls stream state from Redis and resumes RTMP push.",
            "Assert HLS client player recovers playback within 5 seconds without visible buffer starvation."
          ],
          edgeCase: "Primary RTMP endpoint remains blocked after crash, forcing automatic fallback to secondary RTMP
ingress server."
        },
        {
          id: "TC-STRM-02",
          title: "Bitrate Degradation and Frame-Rate Throttle Adaptor",
          severity: "high",
          steps: [
            "Start stream under synthetic network latency of 350ms and 15% packet loss.",
```

```
            "Verify active bitrate reports from FFmpeg encoder process."
          ],
          assertions: [
            "Assert encoder automatically decreases streaming bitrate from 4800 kbps to 2200 kbps.",
            "Assert that resolution scales dynamically from 1080p to 720p.",
            "Verify frame drops are maintained under 5% during throttled connection."
          ],
          edgeCase: "Sudden network recovery to 0% loss; verify encoder scales back up to 1080p within 10 seconds."
        }
      ];
    } else {
      // Default general response
      testCases = [
        {
          id: "TC-GEN-01",
          title: "Functional Flow Validation for: " + prompt.substring(0, 30) + "...",
          severity: "medium",
          steps: [
            `Analyze target endpoints relating to user specification: "${prompt}"`,
            "Run integration tests checking data schemas, authentication headers, and response payload formatting.",
            "Execute visual testing suite checking layouts under responsive constraints."
          ],
          assertions: [
            "Assert endpoints respond with valid JSON payloads and HTTP 200/201.",
            "Assert database fields match input validations precisely.",
            "Verify no UI visual overlap is reported."
          ],
          edgeCase: "Request is dispatched with empty payloads or invalid OAuth bearer tokens."
        }
      ];
    }

    return NextResponse.json({
      success: true,
      suiteName,
      testType,
      generatedAt: new Date().toISOString(),
      prompt,
      testCases
    });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }
}
```